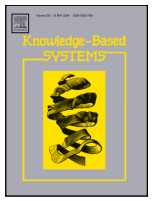
Knowledge-Based Systems 326 (2025) 114065



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





A large language model-driven reward design framework via dynamic feedback for reinforcement learning

Shengjie Sun [a,1,2], Runze Liu [a,1], Jiafei Lyu [a,b], Jing-Wen Yang [c], Liangpeng Zhang [c], Xiu Li [a,*]

[a] Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, 518055, China
[b] TEG, Tencent, Shenzhen, 518057, China
[c] IEG, Tencent, Shenzhen, 518000, China

ARTICLE INFO

Keywords:
Reinforcement learning
Large language model
Reward design

ABSTRACT

Large Language Models (LLMs) have shown significant potential in designing reward functions for Reinforcement Learning (RL) tasks. However, obtaining high-quality reward codes often involves human intervention, numerous LLM queries, or per-iteration RL training. To address these issues, we propose CARD, an LLM-driven Reward Design framework that generates and improves reward function code iteratively. Specifically, CARD includes a Coder that generates and verifies the code, while an Evaluator provides dynamic feedback to improve the code. We introduce Trajectory Preference Evaluation (TPE), which evaluates the current reward function based on trajectory preferences. The Evaluator constructs the feedback based on whether the code passes TPE dynamically, thus avoiding repetitive RL training. Empirical results on Meta-World and ManiSkill2 demonstrate that our method outperforms or matches the baselines across all tasks. On 10 out of 12 tasks, CARD shows better or comparable performance to policies trained with expert-designed rewards and even surpasses the oracle on 3 tasks.

1. Introduction

Reinforcement Learning (RL) has been successfully applied to various tasks with well-defined reward functions [1–3]. Reward functions should encode true indicators of task performance [4]. For example, sparse reward functions indicate whether the task has been successfully completed or not. However, sparse reward signals provide limited feedback, making it difficult to learn a policy. Many RL tasks require reward shaping [5,6] to provide dense reward signals [7–9]. Furthermore, such reward functions do not exist in many real-world scenarios. A common approach is to manually design a reward function, known as reward engineering [10,11], but this requires extensive human knowledge and effort. On the one hand, complex real-world scenarios such as autonomous driving require expert priors. For example, designing reward functions for connected and automated vehicles (CAVs) involves considerations from multiple perspectives carefully designed by experts, including driving safety, traffic efficiency, energy efficiency, and the success of the platoon formed [12,13]. On the other hand, recent studies suggest that rewards designed through manual trial-and-error are probably sub-optimal [14] and may result in unexpected outcomes [15]. To tackle this problem, prior work has explored learning reward functions via inverse RL [16–20] and preference-based RL [21–24]. However, inverse RL requires high-quality demonstrations and preference-based RL often needs a large number of preference labels, both of which limit their use in practical applications.

Recently, Large Language Models (LLMs) have been demonstrated to be effective in generating reward function code for RL tasks [25–28]. However, generating an optimal reward function with a single query remains difficult, and the reward code often requires refinement. Existing methods either rely on human feedback or incur high costs to improve the quality of generated reward functions. For example, Yu et al. [25] depends on manually designed Motion Description templates and reward APIs, with the reward quality directly influenced by the accuracy of these components. Xie et al. [26] requires humans to analyze agent trajectories to enhance the reward function, while Li et al. [28] uses LLMs for this analysis. Ma et al. [27] identifies the most effective reward function using ground-truth rewards after repeated experiments, significantly increasing token consumption and RL training costs. Reducing

* Corresponding author.
E-mail addresses: sunsj23@mails.tsinghua.edu.cn (S. Sun), lrz23@mails.tsinghua.edu.cn (R. Liu), dmksjfl@gmail.com (J. Lyu), jingwenyang@tencent.com (J. Yang), gustavzhang@tencent.com (L. Zhang), li.xiu@sz.tsinghua.edu.cn (X. Li).
[1] Equal contribution.
[2] Work done during an internship at Tencent.

https://doi.org/10.1016/j.knosys.2025.114065
Received 23 February 2025; Received in revised form 22 June 2025; Accepted 4 July 2025
Available online 8 July 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Comparison of LLM-based reward design methods.

Method	Iterative improvement	Human feedback	RL per iteration	Evaluator feedback	# of LLM queries per iteration	# of iterations
L2R [25]	✗	✗	✗	✗	1	0
Text2Reward [1] [26]	✓	✓	✓	✗	1	0
Eureka [27]	✓	✗	✓	✗	16	5
CARD	✓	✗	✗	✓	1	2

[1] Text2Reward supports both reward iterative improvement with additional human feedback and single-flow reward generation without human intervention.

human involvement, token usage, and per-iteration RL training while maintaining task performance remains a great challenge in LLM-based reward design.

To address these issues, we propose an LLM-based **C**oder-Ev**A**luator **R**eward **D**esign framework, named CARD. CARD consists of an LLM-based Coder to generate reward function code and an Evaluator to evaluate the quality of the code. CARD can automatically design the reward functions throughout the entire process without human intervention. Through reward introspection, CARD can timely detect whether it has deviated from the optimization objective and skip unnecessary RL training. This reduces token consumption and RL training costs, resulting in more efficient iterative improvement of reward functions while ensuring similar or better performance. The Coder queries the LLM using environment description and task goals to generate the initial reward function code, which is then verified for successful execution. The Evaluator dynamically provides feedback based on the quality of the reward function, without requiring human involvement or additional LLM queries. The Coder iteratively improves the reward function based on the provided feedback. We introduce Trajectory Preference Evaluation (TPE), which evaluates the reward function with trajectory preferences, allowing the Evaluator to assess reward functions without RL training. If the reward function does not pass TPE, preference feedback is provided to improve the code, while process and trajectory feedback are provided otherwise. Specifically, process feedback records changes in parameters, such as trajectory return and sub-rewards during RL training. This allows the Coder to better understand the overall trends of the training process and evaluate the effectiveness of sub-reward components. Trajectory feedback provides parameter details for each step of successful and failed trajectories, enabling the Coder to compare the impact of sub-reward components on these trajectories. Preference feedback evaluates the reward function using trajectory pairs and preferences via TPE without RL training results. The results show that CARD can design higher quality reward functions without human involvement, outperforming or matching the baselines and even surpassing the human oracle in some tasks. More importantly, CARD reduces token consumption during iterative improvements compared to the previous method [27] and avoids per-iteration RL training, further reducing the cost of reward design. A comprehensive comparison between CARD and previous methods is listed in Table 1. Our code is available at: https://github.com/ShengjieSun419/CARD. The contributions of this paper can be summarized as follows:

- We propose an LLM-based reward design framework that generates and iteratively refines reward code without human feedback or parallel LLM queries.
- We propose Trajectory Preference Evaluation, enabling the Evaluator to provide dynamic feedback for reward function improvement, eliminating per-iteration RL training.
- Empirical results on multiple tasks of Meta-World and ManiSkill2 show that CARD outperforms the baselines and even exceeds the human oracle.

2. Related work

Reward Design. Reward engineering remains a significant challenge in RL [4,10,11,29]. The quality of the reward function is crucial for RL algorithms, especially for applications in real-world scenarios. Numerous research works investigate the construction of high-quality reward functions from diverse perspectives. In Imitation Learning (IL), Ziebart et al. [16], Wulfmeier et al. [17], Finn et al. [18], Ho and Ermon [19], Fu et al. [20] extract a reward function from expert demonstrations via inverse RL. In offline IL setting, Luo et al. [30] obtains rewards via optimal transport. Lyu et al. [31] utilizes a search-based method to design a reward function. These methods require expensive expert demonstrations. Preference-based RL learns from preference relations to obtain reward functions [21–24,32]. However, preference-based RL often requires a large number of preference labels, which is expensive and time-consuming for human experts to provide. CARD differs from the above methods, as it does not require high-quality demonstrations or preference data. Instead, CARD leverages the impressive comprehension and generation capabilities of LLM to automatically generate and enhance reward functions without human costs.

Large Language Models for Reinforcement Learning. Many previous studies attempt to use LLM to assist in RL training. Ma et al. [33], Fan et al. [34], Ma et al. [35], Du et al. [36], Karamcheti et al. [37], Du et al. [38], Kwon et al. [39] utilize pre-trained foundation models to generate reward signals for RL tasks. However, the agent needs a large number of samples from the environment during the learning process. Frequent queries to the LLM not only decrease training efficiency but also lead to significant token consumption. Additionally, most of these methods generate scalar reward values, which are difficult to interpret and improve. Other approaches focus on solving robot control [40–42] and goal planning [43–49] problems by generating clear and structured program code. Most of these works focus on executing robotic actions based on known motion primitives instead of learning low-level skills. Recent studies [25–28] address the challenge of robotic low-level manipulation in RL by creating reward functions using LLMs. However, these methods require human knowledge or lead to significant token consumption and per-iteration RL training. In contrast, CARD can generate reward function code and automatically provide feedback on its quality without human involvement or LLM analysis. Additionally, CARD utilizes TPE to eliminate unnecessary RL training, which further reduces the cost.

3. Problem setup

We consider the standard RL setting, which is formulated as a Markov Decision Process (MDP). The MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ is the transition dynamics, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The agent interacts with the environment at discrete time steps. At each time step t, the agent observes the state s_t, selects an action a_t based on the policy π, and receives a reward r_t. The objective

of the agent is to maximize the expected return $G = \sum_{t=0}^{T} \gamma^t r_t$, where T is the horizon.

4. Method

In this section, we present CARD, which generates and refines reward function code using a Coder and an Evaluator through an iterative process. CARD consists of three main steps: (i) **Reward Design** (▷ Section 4.1): The Coder generates the reward function code based on the environment descriptions and task goals, followed by a code execution check; (ii) **Reward Introspection** (▷ Section 4.2): The Evaluator provides various types of feedback based on the quality and effectiveness of the trained policy in a fully automatic way; and (iii) **Reward Improvement** (▷ Section 4.3): The Coder refines the reward function code based on dynamic feedback, forming an iterative refinement manner. The overall framework of CARD is illustrated in Fig. 1, and the pseudocode of the algorithm is provided in Appendix A.

4.1. Reward design

Code Generation. Generating domain-specific reward functions requires prior knowledge of both the environment and the agent's configuration. To address this, the Coder is provided with an environment description in a Pythonic style [26,27], as shown in the system prompt in Fig. 1. The Coder is also given a task description and a reward function template to guide the LLM in generating task-specific reward function code. The Coder further specifies that the generated reward function should explicitly define each sub-reward [27], rather than simply returning a scalar reward. This encourages the LLM to generate a more comprehensive reward code that accounts for multiple factors, beyond just the distance between the agent and the task target. Further details of the prompts are provided in Appendix E.

Code Execution Check. The reward function code generated by the LLM may contain syntax errors (e.g., undefined variables) or runtime errors (e.g., variable type mismatches or matrix dimension mismatches). One approach to address this is to provide the LLM with error information to correct the code [50,51], but this can alter the original logic. Another approach is to generate multiple reward functions in parallel, increasing the probability that at least one is correct [27]. However, a large number of queries leads to extensive token usage, and determining the optimal parallel number is difficult. In contrast, CARD uses a sequential check mechanism. The agent interacts with the environment using the current reward function, and the Coder checks whether the code is executable or not. If it fails, the Coder generates a new reward function, repeating the process until the code runs successfully. This approach ensures that the correct code is found with minimal token consumption. The execution error rate and token usage for different algorithms are further discussed in Section 5.3 and Appendix C.1.

4.2. Reward introspection

Obtaining the optimal reward function in a single query is challenging, and the generated code often requires further refinement. Previous studies explore three approaches to improve the code: (i) using LLM feedback to refine the reward function [28], although this increases token consumption; (ii) refining the code with human feedback [26], which incurs high manual costs; and (iii) selecting the best reward function from a set of independent experiments [27], but this requires the ground-truth reward and more time for RL training.

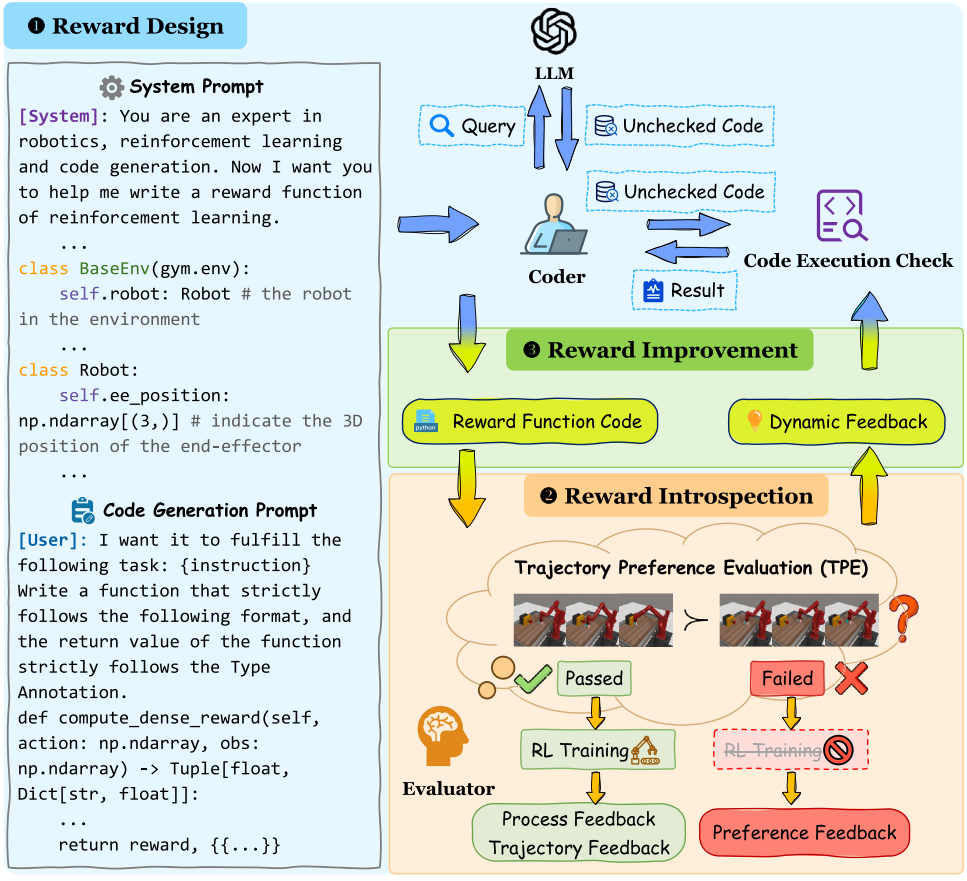


Fig. 1. The framework of CARD. CARD includes an LLM-based Coder to generate reward function code and an Evaluator to evaluate the quality of the code. The reward function is refined iteratively with dynamic feedback from the Evaluator.

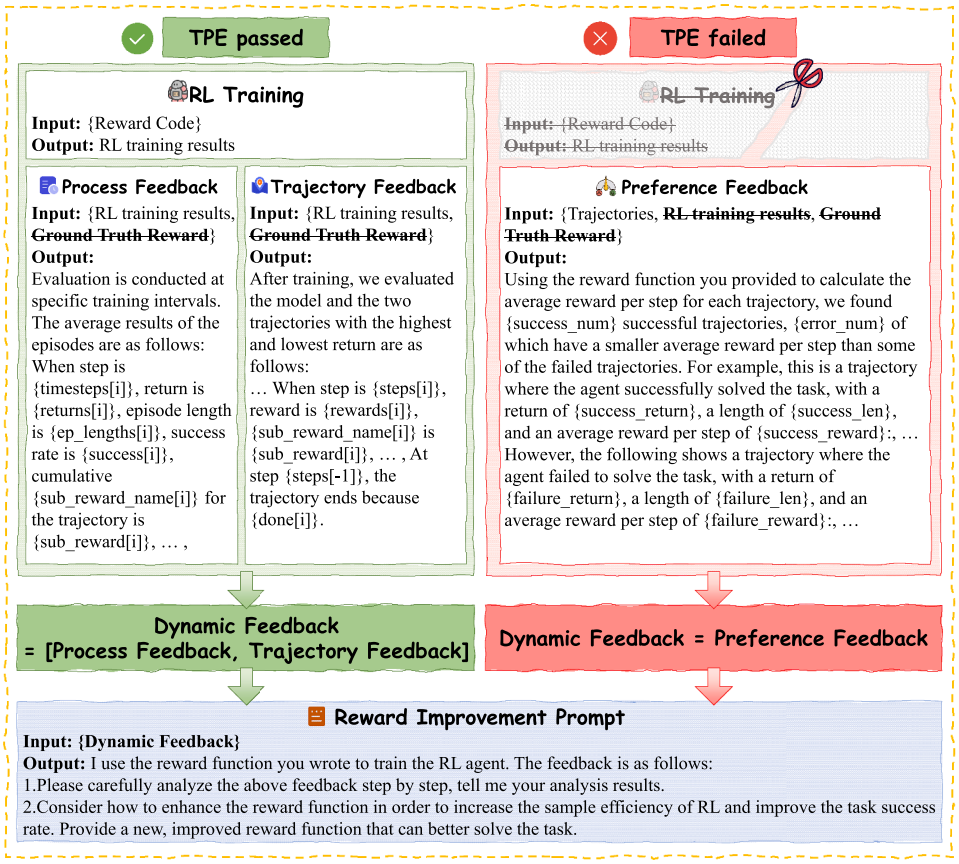


Fig. 2. Construction of three types of feedback and dynamic feedback mechanism.

To address these challenges, CARD introduces an automated approach to construct various types of feedback, including process feedback, trajectory feedback, and preference feedback, as shown in Fig. 2. These feedback types are constructed based on RL training results or historical trajectories and evaluate the reward function from macro, micro, and preference perspectives, respectively. Notably, this process is fully automated and cost-effective, as it avoids additional LLM queries, human involvement, or RL training for each generated reward function.

Process Feedback. During the RL training process, the policy is evaluated at each evaluation step by computing the average return, average trajectory length, average success rate, and sub-rewards. Since the training curves reflect the quality of the reward function, these results can be used as macro feedback to assess the reward function, as proposed in Eureka [27]. Compared to sparse final evaluation results, these dense indicators are more useful for the Coder to understand how the current reward function influences the training process and to identify areas for adjustment, such as reducing the weight of ineffective sub-rewards.

Trajectory Feedback. From a trajectory perspective, we construct trajectory feedback by analyzing sampled trajectories. Comparing reward values and states across trajectories is valuable for improving the reward function [28]. Specifically, the Evaluator uses the trained policy to roll-out trajectories and select those with the highest and lowest returns. In addition, the Evaluator collects detailed information about these selected trajectories, including returns and success flags, while also sampling reward values, sub-reward values, and various observation parameters at intervals. Unlike Li et al. [28], the trajectory feedback is constructed offline without relying on an LLM-based Trajectory Analyzer, which improves the efficiency of reward introspection and reduces token costs.

Trajectory Preference Evaluation. Process feedback and trajectory feedback are both based on RL training results. However, training RL agents in each reward improvement iteration is computationally expensive, and ground-truth rewards are required to evaluate the improvements [27]. To address these issues, we introduce Trajectory Preference Evaluation (TPE), which evaluates the reward function using trajectory preferences without requiring ground-truth rewards or time-consuming RL training. The key insight behind TPE is that successful trajectories are preferred over failed ones, and these preference relations can naturally serve as signals to improve the reward function. In recent preference-based RL algorithms [23,24], preferences act as direct supervised signals for learning the reward model, and we adopt a similar approach by using relative preferences for reward improvement. We begin by providing a formal definition of an order-preserving reward function and then describe how preference feedback is constructed.

In many goal-conditioned environments [7,52], episodes end when the task is completed. As a result, successful trajectories are typically shorter than failed ones, meaning that failed trajectories, which take more steps, can have higher returns than successful ones. To address this issue, we use the average return per step as the metric and define an order-preserving reward function as follows:

Definition 4.1. *Given a set of trajectories* $\tau = \{\tau^1, \tau^2, \ldots, \tau^N\}$*, where each trajectory* $\tau^k = \{(s_0^k, a_0^k), \ldots, (s_{T_k}^k, a_{T_k}^k)\}$ *and* T_k *is the horizon of* τ^k*. Let* $\tau^+ \subseteq \tau$ *be the subset of successful trajectories, and* $\tau^- \subseteq \tau$ *be the subset of failed trajectories. A reward function* $r(s, a)$ *is said to be order-preserving if it satisfies the following condition:*

$$\min_{\tau^i \in \tau^+} \frac{G^i}{T^i + 1} > \max_{\tau^j \in \tau^-} \frac{G^j}{T^j + 1}, \tag{1}$$

where $G^k = \sum_{t=0}^{T^k} \gamma^t r(s_t^k, a_t^k), k \in \{i, j\}$.

This definition ensures that the successful trajectory with the lowest average return per step has a higher value than the failed trajectory with the highest average return per step. This ordering creates a clear separation between successful and failed trajectories in terms of their average returns, regardless of trajectory length.

Preference Feedback. To construct preference pairs, we collect both successful and failed trajectories using the trained policy. After performing TPE on these trajectories, we generate preference feedback that includes the proportion of successful trajectory returns that exceed those of failed trajectories. As shown in Fig. 2, this feedback also provides details of one successful trajectory and one failed trajectory in the trajectory feedback format. More information on the three types of feedback is available in Appendix E.

4.3. Reward improvement

Dynamic Feedback. With TPE, the Evaluator can pre-assess the current reward function r using historical trajectories before RL training. As shown in Fig. 2, if the reward function is order-preserving, RL training is performed, and both process feedback and trajectory feedback are obtained. If r does not align with the expected preferences, RL training is skipped, and only preference feedback is constructed. Let F_{proc} be the process feedback, F_{traj} be the trajectory feedback, and F_{pref} be the preference feedback. The dynamic feedback F is defined as:

$$F = \begin{cases} [F_{\text{proc}}, F_{\text{traj}}] & \text{if } r \text{ is order-preserving,} \\ F_{\text{pref}} & \text{otherwise,} \end{cases} \tag{2}$$

where $[\cdot, \cdot]$ denotes the concatenation of two types of feedback. This dynamic feedback mechanism eliminates the need for RL in each iteration and significantly improves the efficiency of reward introspection.

For reward improvement, the Coder prompts the LLM with dynamic feedback and the chat history, and the process continues with further reward introspection and improvement. The LLM iteratively refines the code with the provided context, and the process is terminated once the maximum number of iterations is reached. Refer to Section 5.4 for further discussion on the number of iterations.

5. Experiments

In this section, we conduct experiments to answer the following questions: (**Q1**) How does CARD perform on the evaluation tasks compared to the baselines? (**Q2**) How does CARD perform with respect to token efficiency? (**Q3**) How sensitive is CARD to its key parameters and components? (**Q4**) How does CARD improve the reward function with dynamic feedback?

Additionally, we discuss two other issues in Appendix C: (**Q5**) How does CARD perform in terms of the execution error rate of the generated reward function code? (**Q6**) How does CARD perform when using different LLM APIs?

5.1. Experimental setup

Following Xie et al. [26], we evaluate on 6 robotic manipulation tasks of Meta-World [7] and 6 robotic manipulation tasks of ManiSkill2 [52]. Detailed descriptions of these tasks are provided in Appendix B.1.

Baselines. To evaluate the performance of CARD, we compare it against the following baselines: (i) **Oracle**: The reward function is designed by human experts. (ii) **L2R** [25]: L2R generates reward functions by combining human-defined sub-reward components. (iii) **Text2Reward** [26]: Text2Reward generates reward function code using zero-shot or few-shot methods by providing a textual description of the environment and task instructions. (iv) **Eureka** [27]: Eureka employs an evolutionary search strategy to improve the reward function by selecting the best-performing one from 16 independent trials.

For a fair comparison, all experiments are conducted in a zero-shot setting and `GPT-4-1106-preview` [53] API is used for all methods. For L2R, we provide detailed task instructions for the Motion Descriptor and organize oracle reward components into separate reward APIs for the Reward Coder. This allows the LLM to perceive almost all relevant environment details, enabling L2R to perform similarly to oracle. For other methods, we provide the same environment abstraction as Xie et al. [26]. Therefore, the range of variables that can be used when generating the reward function is the same, ensuring the fairness of the comparison. For Text2Reward, we use both the human-improved code provided by Xie et al. [26] and the generated code without human feedback (denoted as **Reproduce** in Section 5.2). For Eureka, we generate the initial reward code and refine it for 5 iterations, with each iteration consisting of 16 parallel samplings. We apply the same RL algorithm (PPO or SAC) and set the same hyperparameters as Text2Reward [26] across all methods. For CARD, we set δ to 0.8 and run 2 iterations to refine the reward function code. Further details about CARD are provided in Appendix B.2. Other parameter settings are reported in Appendix B.3.

5.2. Results (Q1)

As shown in Figs. 3 and 4, our observations (Obs.) are as follows: **Obs.❶ CARD consistently achieves better or similar performance than other baselines on all tasks, while it is also more cost-efficient.** Notably, on the *Sweep Into* task, CARD demonstrates significant improvement compared to the baselines. **Obs.❷ CARD is comparable to or even better than expert-designed rewards.** On 3 out of 12 tasks, CARD even surpasses oracle. For the remaining tasks, CARD still performs comparably to oracle. We hypothesize that the reward function code generated by CARD is highly effective given the available information, leaving minimal room for further improvement.

5.3. Token efficiency (Q2)

To further compare the cost of LLM queries of different algorithms, we compute the total input and output tokens consumed. L2R queries the LLM once using the Motion Descriptor and once using the Reward Coder, so we sum the cost of both queries as the overall cost. Text2Reward, without human feedback improvement, performs only one LLM query. Eureka generates the initial reward code and refines it for 5 iterations and each generation contains 16 parallel sampling. Therefore, the token usage of Eureka is computed by the sum of 96 queries. For CARD, we measure the token consumption over initial generation and 2 rounds of reward introspection, which requires 3 LLM queries. In our experiments, token usage is calculated from 16 independent runs for both L2R and Text2Reward, and we report the average values. Since both Eureka and CARD involve iterations, we measure token consumption from one run due to cost limitations.

Table 2 summarizes the average token consumption of different algorithms for LLM queries across all evaluation tasks. We offer the following key observations: **Obs.❸ CARD strikes a balance between token consumption and performance.** Although Text2Reward without iterative improvement consumes the least tokens, it still incurs additional token consumption and high labor costs when human feedback is introduced. L2R also consumes a small number of tokens because LLM only needs to select the reward API without requiring details of the environment settings. However, human efforts are needed to implement the reward API. Eureka consumes the most tokens, as it improves the reward function through parallel queries iteratively. CARD consumes fewer tokens than other iterative methods while achieving better performance than non-iterative methods. Complete token consumption details for all algorithms are provided in Appendix C.3.

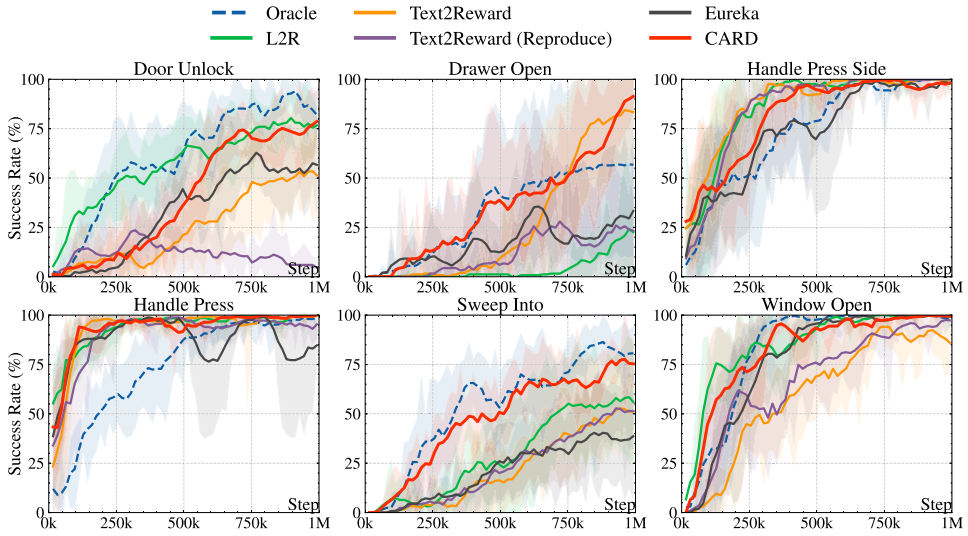


Fig. 3. Learning curves on six **Meta-World** tasks, measured by task success rate. The solid line represents the mean success rate, while the shaded regions correspond to the standard deviation, both calculated across five random seeds.

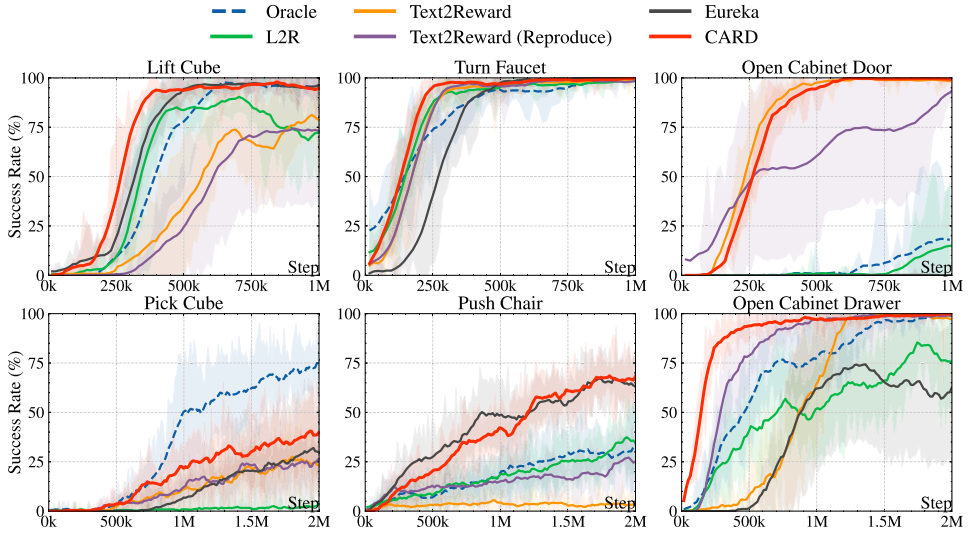


Fig. 4. Learning curves on six **ManiSkill2** tasks, measured by task success rate. The solid line represents the mean success rate, and the shaded areas denote the standard deviation, calculated across five random seeds.

Table 2
Comparison of token usage among different methods averaged across all evaluation tasks.

Method	Token consumption
L2R	2310.7
Text2Reward	1724.1
Eureka	662882.5
CARD	14181.7

5.4. Ablation study (Q3)

Different numbers of iterations. To investigate how different numbers of iterations influence CARD, we conduct experiments with iterations of $\{1, 2, 3\}$. With the results illustrated in Fig. 5, we observe that: **Obs.❹ The quality of the reward function improves during the initial iterations and then gradually converges.** In particular, on 7 out of 9 tasks, the results after the second iteration are either the best or comparable to those of the other iterations. Significant improvements are only observed in the third iteration for the more challenging tasks, such as *Door Unlock* and *Sweep Into*. However, on *Handle Press Side* and *Turn Faucet*, performance declines considerably after the third iteration. We hypothesize that after generating a well-shaped reward function, further refinement by the LLM may introduce unwanted changes, leading to a drop in performance. Therefore, for more difficult tasks, increasing the number of iterations is beneficial for fully refining the reward function, whereas for simpler tasks, limiting the number of refinement iterations may be more effective.

Different types of feedback. To analyze the impact of different types of feedback on CARD, we conduct experiments on three tasks (*Door Unlock*, *Drawer Open* and *Handle Press Side*) of Meta-World by removing process feedback, trajectory feedback, and preference feedback, respectively. As

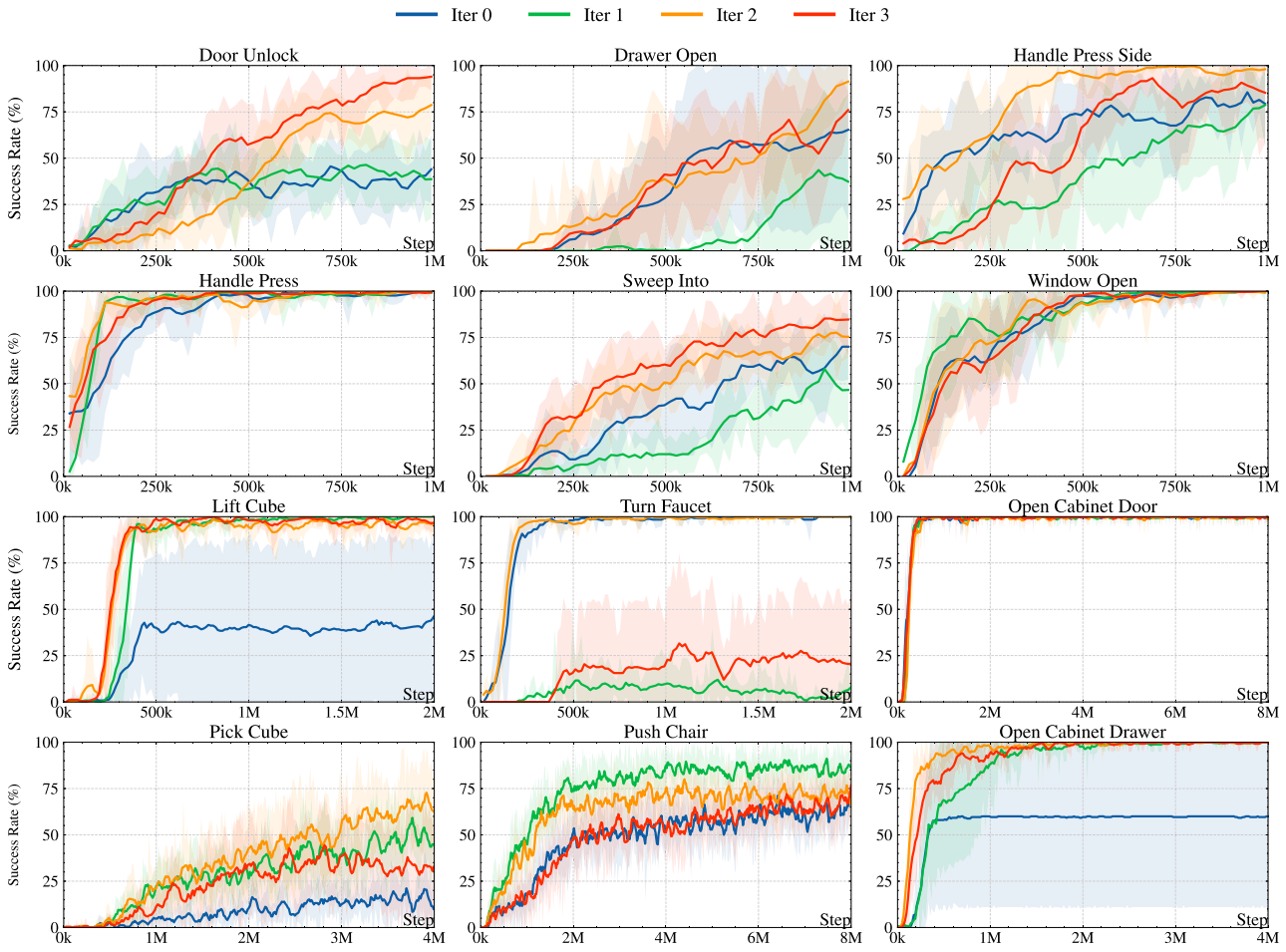


Fig. 5. Learning curves of CARD on **Meta-World** and **ManiSkill2** tasks with different numbers of iterations. The solid line represents the mean success rate, and the shaded areas denote the standard deviation, calculated across five random seeds.

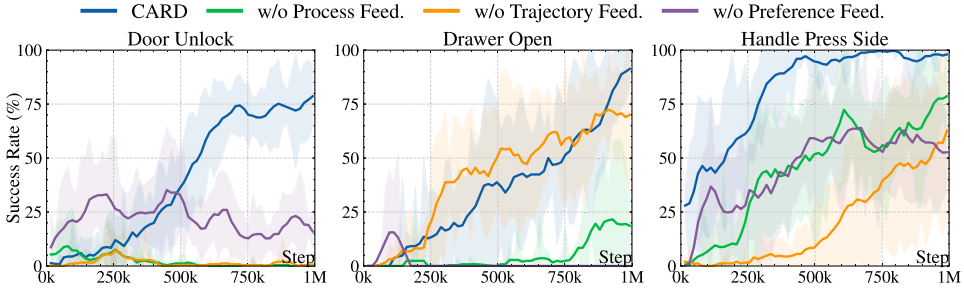


Fig. 6. Learning curves of CARD on three **Meta-World** tasks with different types of feedback. The solid line represents the mean success rate, and the shaded areas denote the standard deviation, calculated across five random seeds.

shown in Fig. 6, we observe that: **Obs.❺ CARD achieves the best performance with all types of feedback.** Removing either feedback can cause the performance to drop significantly.

Sensitivity Analysis of TPE. The curve labeled "w/o Preference Feed." in Fig. 6 shows the performance of CARD without preference feedback and TPE. The results indicate that: **Obs.❻ Constructing preference feedback using TPE can enhance the performance, with the improvement being more noticeable on more complex tasks.** Specifically, using TPE to construct preference feedback on the simpler task (*Handle Press Side*) improves performance, and the improvement is more obvious on more complex tasks (*Door Unlock* and *Drawer Open*). We attribute this to the fact that, when designing reward functions for complex tasks, LLM is more likely to make errors or deviate from the optimal reward.

5.5. Case study (Q4)

To understand how different types of feedback iteratively improve the reward function code through reward introspection, we present a case study on the *Door Unlock* task. In the case study, we aim to illustrate how CARD guides the improvement of the reward function

```
@@ -1,43 +1,41 @@

 from typing import Tuple, Dict
 import numpy as np

 def compute_dense_reward(self, action: np.ndarray, obs: np.ndarray) -> Tuple[float, Dict[str, float]]:
     # Constants for reward calculation
-    DISTANCE_WEIGHT = -1.0
-    ORIENTATION_WEIGHT = -0.5
-    ACTION_REGULARIZATION_WEIGHT = -0.1
-    GRIP_REWARD_WEIGHT = 0.5
-    ROTATION_REWARD_WEIGHT = 1.0
+    DISTANCE_WEIGHT = -0.5  # Reduced impact to balance with other rewards
+    ACTION_REGULARIZATION_WEIGHT = -0.01  # Reduced impact as it was minor
+    GRIP_REWARD_WEIGHT = 1.0  # Increased impact to be significant when close to the object
+    ROTATION_REWARD_WEIGHT = 2.0  # High impact for the main task
+    ROTATION_THRESHOLD = 0.1  # Threshold for considering a successful rotation

     # Initialize reward and components dictionary
     reward = 0.0
     reward_components = {
         "distance_reward": 0.0,
-        "orientation_reward": 0.0,
         "action_regularization": 0.0,
         "grip_reward": 0.0,
         "rotation_reward": 0.0
     }

     # Calculate distance reward
     distance = np.linalg.norm(obs[:3] - obs[4:7])
     reward_components["distance_reward"] = DISTANCE_WEIGHT * distance

-    # Orientation reward (not implemented, as we have no target orientation for the lock)
-    # reward_components["orientation_reward"] = ...
-
     # Action regularization (reward for using smaller actions)
     reward_components["action_regularization"] = ACTION_REGULARIZATION_WEIGHT * np.sum(np.square(action))

     # Grip reward (if the gripper is close enough to the lock, and the openness is appropriate)
     if distance < 0.05:  # Threshold for "close enough"
         reward_components["grip_reward"] = GRIP_REWARD_WEIGHT * (1.0 - abs(obs[3]))

-    # Rotation reward (if the lock is being rotated counter-clockwise, not implemented as we don't have the
-    # reward_components["rotation_reward"] = ...
+    # Comment: Implement rotation reward based on the change in the lock's orientation, assuming counter-clo
+    # This is a placeholder implementation and would need the actual logic for determining the rotation dire
+    if obs[7:11][0] > ROTATION_THRESHOLD:
+        reward_components["rotation_reward"] = ROTATION_REWARD_WEIGHT * obs[7:11][0]

     # Sum up the total reward
     reward = sum(reward_components.values())

     return reward, reward_components
```

Fig. 7. Comparison of reward function code before and after iteration 1 reward introspection by CARD on *Door Unlock*.

across two iterations. In the first iteration, the Coder modifies the reward function based on process feedback and trajectory feedback. These forms of feedback help the Coder adjust the coefficients of different sub-rewards and remove invalid sub-rewards. In the second iteration, TPE is applied. TPE reveals that the reward function is misaligned with the task goal, as successful trajectories should yield a higher return than failed ones. Based on preference feedback from TPE, the Coder realizes this misalignment and makes a targeted improvement by boosting the reward for completing the task and modifying the existing sub-rewards, which leads to better alignment with the task objectives and trajectory preferences. Additionally, we provide a detailed analysis of a failure case in Appendix C.4. We also provide an example in Appendix C.5 where CARD performs better than the oracle.

Iteration 1. Fig. 7 illustrates how CARD improves the reward code during the first iteration of reward introspection. During this iteration, the Coder receives process feedback and trajectory feedback from RL training results. It can be seen that the LLM implements `rotation_reward` based on the feedback and adjusts the weights for each reward component. Additionally, the Coder removes `orientation_reward`, which is not implemented. From the results in Fig. 5, it can be observed that this refinement has no effect.

Iteration 2. In the next iteration, the LLM incorporates preference feedback, recognizing that successful trajectories should yield a higher return than failed ones. Fig. 8 shows that the Coder introduces a `success_reward`, providing a large reward for task completion. The `grip_reward` is refined to include a base reward and a bonus to encourage the gripper to close. The `rotation_reward` is removed, as it did not improve the results. After this iteration, the policy's success rate increases significantly, indicating that the reward function is now better aligned with trajectory preferences and task goals.

```
@@ -1,41 +1,47 @@

 from typing import Tuple, Dict
 import numpy as np

 def compute_dense_reward(self, action: np.ndarray, obs: np.ndarray) -> Tuple[float, Dict[str, float]]:
     # Constants for reward calculation
-    DISTANCE_WEIGHT = -0.5  # Reduced impact to balance with other rewards
-    ACTION_REGULARIZATION_WEIGHT = -0.01  # Reduced impact as it was minor
-    GRIP_REWARD_WEIGHT = 1.0  # Increased impact to be significant when close to the object
-    ROTATION_REWARD_WEIGHT = 2.0  # High impact for the main task
-    ROTATION_THRESHOLD = 0.1  # Threshold for considering a successful rotation
-
+    DISTANCE_WEIGHT = -0.1  # Reduced impact to allow other rewards to play a role
+    ACTION_REGULARIZATION_WEIGHT = -0.01  # Keep this small to avoid discouraging exploration
+    GRIP_REWARD_BASE = 0.5  # Flat reward for gripping correctly
+    GRIP_REWARD_SCALE = 0.5  # Scale for grip reward based on gripper openness
+    SUCCESS_REWARD = 10.0  # Large reward for successful completion of the task
+

     # Initialize reward and components dictionary
     reward = 0.0
     reward_components = {
         "distance_reward": 0.0,
         "action_regularization": 0.0,
         "grip_reward": 0.0,
-        "rotation_reward": 0.0
+        "success_reward": 0.0
     }

     # Calculate distance reward
     distance = np.linalg.norm(obs[:3] - obs[4:7])
     reward_components["distance_reward"] = DISTANCE_WEIGHT * distance

     # Action regularization (reward for using smaller actions)
     reward_components["action_regularization"] = ACTION_REGULARIZATION_WEIGHT * np.sum(np.square(action))

-    # Grip reward (if the gripper is close enough to the lock, and the openness is appropriate)
-    if distance < 0.05:  # Threshold for "close enough"
-        reward_components["grip_reward"] = GRIP_REWARD_WEIGHT * (1.0 - abs(obs[3]))
-
-    # Comment: Implement rotation reward based on the change in the lock's orientation, assuming counter-clock
-    # This is a placeholder implementation and would need the actual logic for determining the rotation direct
-    if obs[7:11][0] > ROTATION_THRESHOLD:
-        reward_components["rotation_reward"] = ROTATION_REWARD_WEIGHT * obs[7:11][0]
+    # Grip reward (encouraging the robot to grip when close to the object)
+    # Comment: Reward is now given for attempting to grip, scaled by how close the gripper is to the correct c
+    is_close = distance < 0.05  # Threshold for "close enough"
+    reward_components["grip_reward"] = GRIP_REWARD_BASE if is_close else 0.0
+    reward_components["grip_reward"] += GRIP_REWARD_SCALE * (1.0 - abs(obs[3])) if is_close else 0.0
+
+    # Comment: Implement success reward
+    # Success reward is given if the task is completed successfully
+    # Here we infer success if the object position is close to the goal position
+    # and the gripper is in a state indicative of "holding" (e.g., not fully open or closed)
+    goal_reached = np.linalg.norm(obs[4:7] - self.env._get_pos_goal()) < 0.05
+    gripper_holding = 0.2 < obs[3] < 0.8
+    if goal_reached and gripper_holding:
+        reward_components["success_reward"] = SUCCESS_REWARD

     # Sum up the total reward
     reward = sum(reward_components.values())

     return reward, reward_components
```

Fig. 8. Comparison of reward function code before and after iteration 2 reward introspection by CARD on *Door Unlock*.

6. Conclusion

In this paper, we propose CARD, an LLM-driven reward design framework that iteratively generates and refines reward function code without relying on redundant LLM queries or human feedback. The framework includes Coder, which is responsible for generating reward functions, and Evaluator, which evaluates these functions using dynamic feedback. In CARD, we introduce Trajectory Preference Evaluation, which enables Evaluator to assess the reward functions without requiring RL training at every iteration, thus improving efficiency. The Evaluator provides multiple types of feedback, which guide the Coder in refining the reward code, creating a self-improving and automated process. Empirical results demonstrate that CARD outperforms or matches the baselines on most tasks. Additionally, it even surpasses the oracle on some tasks, demonstrating its strong capability in reward design for robotic manipulation tasks.

CARD focuses on RL problems related to robotic tasks, following prior foundational work [25–27]. However, we believe CARD has strong

potential to generalize to more diverse RL environments, including non-robotic tasks like games (e.g., Gym Atari). We explain this potential in detail from three perspectives:

- In the Reward Design stage, CARD operates in a zero-shot setting, meaning that it designs rewards without few-shot or fine-tuned models. This demonstrates that CARD can design appropriate reward functions for other RL environments, given only the environment description and task instructions.
- In the Reward Introspection stage, the feedback construction in CARD relies on RL training results and historical trajectories. Process feedback is built from training results (e.g., average returns, average trajectory length, average success rate, and sub-rewards). Trajectory feedback is derived from training results (e.g., returns, success flags, reward values, sub-reward values, and observation parameters). Preference feedback and TPE use historical trajectories (e.g., state-action pairs). These types of data are common across nearly all RL environments, whether robotic or non-robotic. Therefore, CARD can generate feedback in a similar approach for other environments.
- In the Reward Improvement stage, the dynamic feedback process based on TPE results is flexible and can be applied to various RL tasks, not just robotic ones.

For future work, we consider utilizing Vision-Language Models (VLMs) to generate reward functions with both vision and language information. Additionally, we will explore extending CARD to handle more diverse environments and tasks.

7. Limitations

Our research demonstrates that LLMs can effectively improve dense functions in RL without human feedback. We design three feedback types that allow the LLM to generate reward functions with sub-rewards and perform introspection using available data. As shown in Table C.11, our method's token consumption increases linearly with each iteration. This means that tasks requiring many iterations to refine reward codes will use a large number of tokens. As previous LLM-based reward design methods do [25–27], our method assumes prior knowledge of environmental information. Another possible limitation is that some environments do not provide explicit indicators of success or failure, which makes it impossible to construct preferences relations through the TPE. However, since most real-world tasks include clear criteria for success or failure, CARD remains applicable in a broad set of scenarios. We do not identify any risks to the best of our knowledge.

CRediT authorship contribution statement

Shengjie Sun: Writing – original draft, Validation, Software, Methodology, Conceptualization; **Runze Liu:** Writing – original draft, Validation, Software, Methodology, Conceptualization; **Jiafei Lyu:** Writing – review & editing, Methodology, Conceptualization; **Jing-Wen Yang:** Writing – review & editing, Resources, Conceptualization; **Liangpeng Zhang:** Resources, Conceptualization; **Xiu Li:** Supervision, Resources, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the STI 2030-Major Projects under Grant 2021ZD0201404.

Appendix A. Algorithm

The full algorithm of CARD is presented in Algorithm 1.

Algorithm 1 CARD.

Input: Environment description M, task instruction l, chat model in LLM $Coder$, offline text formatter $Evaluator$, system prompt P_s, code generation prompt P_i, process feedback template P_{proc}, trajectory feedback template P_{traj}, preference feedback template P_{pref}, Trajectory Preference Evaluation function F_p, reinforcement learning training function F_t, chat history $\mathcal{L}$, trajectories $\mathcal{T}$, iteration number N

$\mathcal{L} = \{P_s(M), P_i(l)\}$
// Reward Design
$R = \text{Coder}(\mathcal{L})$
result $= F_t(R)$
$\mathcal{T} \leftarrow \mathcal{T} \cup \{\text{result}\}$
for N iteration **do**
  // Reward Introspection
  // Check whether R is order-preserving according to Eq. (1)
  preference $= F_p(R, \mathcal{T})$
  // Construct dynamic feedback
  if preference **then**
    // According to Eq. (2) (Upper part)
    result $= F_t(R)$
    $\mathcal{T} \leftarrow \mathcal{T} \cup \{\text{result}\}$
    $\mathcal{L} \leftarrow \mathcal{L} \cup \{P_{proc}(\text{result}), P_{traj}(\text{result})\}$
  else
    // According to Eq. (2) (Lower part)
    $\mathcal{L} \leftarrow \mathcal{L} \cup \{P_{pref}(\text{preference})\}$
  end if
  // Reward Improvement
  $R = \text{Coder}(\mathcal{L})$
end for

Output: Reward function code R

Appendix B. Experimental details

In this section, we provide hyperparameter details used for reward design, reward introspection, and reward improvement.

B.1. Task details

Meta-World. In experimental evaluation, we use 6 robotic manipulation tasks from Meta-World benchmark [7], including *Door Unlock*, *Drawer Open*, *Handle Press Side*, *Handle Press*, *Sweep Into* and *Window Open*. We provide descriptions of each task as follows:

Table B.3
Instructions for generating reward code of each task in Meta-World.

Task	Instruction
Door Unlock	Unlock the door by rotating the lock counter-clockwise.
Drawer Open	Open a drawer by its handle.
Handle Press Side	Press a handle down sideways.
Handle Press	Press a handle down.
Sweep Into	Sweep a puck from the initial position into a hole.
Window Open	Push and open a sliding window by its handle.

Table B.4
Instructions for generating reward code of each task in ManiSkill2.

Task	Instruction
LiftCube	Pick up cube A and lift it up by 0.2 meter.
PickCube	Pick up cube A and move it to the 3D goal position.
TurnFaucet	Turn on a faucet by rotating its handle. The task is finished when qpos of faucet handle is larger than target qpos.
OpenCabinetDoor	A single-arm mobile robot needs to open a cabinet door. The task is finished when qpos of cabinet door is larger than target qpos.
OpenCabinetDrawer	A single-arm mobile robot needs to open a cabinet drawer. The task is finished when qpos of cabinet drawer is larger than target qpos.
PushChair	A dual-arm mobile robot needs to push a swivel chair to a target location on the ground and prevent it from falling over.

- *Door Unlock*: The objective is to manipulate a robotic arm to unlock the door. The initial position of the door is random.
- *Drawer Open*: The objective is to manipulate a robotic arm to open the drawer. The initial position of the drawer is random.
- *Handle Press Side*: The objective is to manipulate a robotic arm to press the handle down sideways. The initial position of the handle is random.
- *Handle Press*: The objective is to manipulate a robotic arm to press the handle. The initial position of the handle is random.
- *Sweep Into*: The objective is to manipulate a robotic arm to sweep the puck into the hole. The initial position of the puck is random.
- *Window Open*: The objective is to manipulate a robotic arm to open the window. The initial position of the window is random.

The instructions for generating reward code are listed in Table B.3, following Xie et al. [26].

ManiSkill2. We utilize 6 manipulation tasks from ManiSkill2 [52], including *LiftCube*, *PickCube*, *TurnFaucet*, *OpenCabinetDrawer*, *OpenCabinetDoor* and *PushChair*. The descriptions of tasks from ManiSkill2 are as follows:

- *LiftCube*: The objective is to lift the cube to the target position.
- *PickCube*: The objective is to pick up the cube to the target position.
- *TurnFaucet*: The objective is to turn on the handle of the faucet.
- *OpenCabinetDrawer*: The objective is to manipulate the single-arm robot to open the drawer on the cabinet.
- *OpenCabinetDoor*: The objective is to manipulate the single-arm robot to open the door on the cabinet.
- *PushChair*: The objective is to manipulate the dual-arm robot to push the chair to the target position.

The instructions for generating the reward code are listed in Table B.4.

B.2. CARD details

Initially, Evaluator uses the code generated by Coder, which may not be optimal, to conduct RL training and collect trajectories. Process feedback and trajectory feedback are then constructed to allow the Coder to refine the code. Each time Coder improves the reward function, Evaluator conducts TPE on the updated code. If the reward function is order-preserving, RL training is performed, and process feedback and trajectory feedback are provided to Coder. Evaluator also records the generated trajectories for subsequent TPE analysis. If the reward function is not order-preserving, it is considered unreasonable, and RL training is skipped to avoid unnecessary costs. Instead, preference feedback is provided, informing the LLM that the updated reward function may deviate from the optimal.

In implementation, we count the number of successful task trajectories whose average per-step return is greater than that of failed task trajectories to calculate the accuracy of the preference order. If the accuracy exceeds a threshold, the reward function is considered order-preserving. This approach provides flexibility in refining reward functions, allowing ample modification opportunities and ensuring TPE is robust to noisy data. During the first reward introspection, Evaluator conducts RL training directly and collects trajectories, as there are no initial trajectories for TPE evaluation. Starting from the second reward introspection, TPE results determine whether RL training is necessary. TPE significantly reduces unnecessary RL training and shortens the time spent on reward introspection. The results in Section 5.2 show that alternating between process feedback, trajectory feedback, and preference feedback improves the Coder's understanding of the environment, task goals, and reward functions, leading to the development of more effective reward functions.

B.3. Implementation details

Since the `GPT-4-0314` [53] API used by Xie et al. [26] has been deprecated,[1] we use `GPT-4-1106-preview` [53] API in all methods. We use `GPT-3.5-turbo-1106` in Section 5.4 to investigate the impact of different LLM APIs on CARD. To ensure a fair comparison, we set the temperature of sampling to 0.7 for all experiments, as it is the default for Text2Reward [26], and a reasonable compromise between the different baselines. Specifically, L2R [25] uses 0.3 and Eureka [27] uses 1.0. For each generated reward function, CARD uses dynamic execution to detect runtime errors. In the event of an error in the reward function code, Coder initiates a re-query to the LLM. The hyperparameter *max_try_num* serves to prevent excessive costs associated with querying the LLM by either generating an executable code or ceasing the process after *max_try_num* attempts. Throughout our experiments, we set *max_try_num* to 10.

We use the Proximal Policy Optimization (PPO) [54] or Soft Actor-Critic (SAC) [55] algorithm to evaluate the reward function. Specifically, we use the open-source implementations of the above algorithms by Raffin et al. [56] and set the exact same hyperparameters as Xie et al. [26] for all methods. The hyperparameters on different benchmarks are shown in Tables B.5 and B.6. We also provide the hyperparameter settings of CARD, as shown in Table B.7.

Each task is run on a single NVIDIA RTX 3090 GPU with 8 CPU cores. To accelerate the RL training process, we leverage the vectorized environment acceleration provided by Raffin et al. [56]. Consistent with the setting of Xie et al. [26], we use 8 parallel environments for training and 5 parallel environments for evaluation in all experiments. In each evaluation step, 10 episodes are sampled to compute the average task success rate. The final results of all experiments are the average over 5 independent runs with different random seeds. We report the training time for different tasks using the hardware and parameters described above in Table B.8. The time required to perform CARD depends on the number of iterations. In the case of 3 iterations and assuming all other parameters remain the same as above, CARD requires up to 4 RL training runs in the worst case (including the evaluation of the final reward function). Using a single NVIDIA RTX 3090 GPU with 8 CPU cores, all tasks except *PushChair* can be completed within 7 hours each. For each task on Meta-World, the total time is approximately 2 hours. Since TPE probably skips per-iteration RL training, the actual computation time is further reduced. The time spent on LLM queries is significantly shorter than that required for RL training and is therefore ignored. In addition, when using the SAC [55] algorithm, each task on Meta-World requires approximately 2050 MB of GPU memory. When using either the PPO [54] algorithm or the SAC [55] algorithm, each task on ManiSkill2 requires approximately 2600 MB of GPU memory. Overall, CARD

[1] https://platform.openai.com/docs/models

Table B.5
Hyperparameters of PPO algorithm used in *Lift Cube* and *Pick Cube* tasks of ManiSkill2.

Hyperparameters	Value
Learning rate	$3e^{-4}$
# of layers	2
Hidden units per layer	256
# of steps per update	3200
# of epochs per update	15
Batch size	400
Discount factor γ	0.85
Target KL divergence	0.05
# of environments	8
GAE λ	0.95
Clip range	0.2
Rollout steps per episode	100

Table B.6
Hyperparameters of SAC algorithm used in *Turn Faucet*, *Open Cabinet Drawer*, *Open Cabinet Door* and *Push Chair* tasks of ManiSkill2 and all Meta-World tasks.

Hyperparameters	Value
Learning rate	$3e^{-4}$
# of layers	3 (Meta-World), 2 (ManiSkill2)
Hidden units per layer	256
Target update frequency	2 (Meta-World), 1 (ManiSkill2)
Train frequency	1 (Meta-World), 8 (ManiSkill2)
Soft update τ	$5e^{-3}$
Gradient steps	1 (Meta-World), 4 (ManiSkill2)
Learning starts	4000
Batch size	512 (Meta-World), 1024 (ManiSkill2)
Discount factor γ	0.99 (Meta-World), 0.95 (ManiSkill2)
Initial temperature	0.1 (Meta-World), 0.2 (ManiSkill2)
# of environments	8
Rollout steps per episode	500 (Meta-World), 200 (ManiSkill2)

Table B.7
Hyperparameters of CARD.

Hyperparameters	Value
# of trajectories collected per iteration	100
Sampling interval for constructing process feedback	100 (Meta-World) 25 (ManiSkill2)
# of sampling points for constructing trajectory feedback	10
Reward preference threshold δ	0.8

Table B.8
Training steps and average time of each task.

Benchmark	Task	Steps	Average time
Meta-World	All	1 M	0.5 h
ManiSkill2	LiftCube	1 M	0.5 h
	TurnFaucet	1 M	0.9 h
	OpenCabinetDoor	1 M	1.3 h
	PickCube	2 M	1.4 h
	PushChair	2 M	4.1 h
	OpenCabinetDrawer	2 M	2.1 h

has low computational and time costs, making it feasible for real-world deployments. In contrast, manually designing reward functions requires trial-and-error [9], which involves multiple human interventions. Studies [14] have shown that even experts may design suboptimal reward functions, and the outcomes of such designs may deviate from the original intention.

Table C.9
Comparison of execution error rate among different methods averaged across all evaluation tasks.

Method	Code execution error rate (%)
L2R	6
Text2Reward	48
Eureka	17
CARD	13

Appendix C. Additional results

C.1. Code execution error rate (Q5)

To compare the robustness of reward generation across different algorithms, we evaluate the execution error rate of the reward function code designed by LLM. For L2R and Text2Reward, we independently run 16 experiments and assess the execution error rate of the generated reward functions. For Eureka and CARD, we calculate the results based on the first generated reward functions and the improved versions during the iteration process. We calculate the average over all tasks, as shown in Table C.9. We observe that: **Obs.❷ CARD has better code generation robustness.** The results show that L2R achieves the lowest code execution error rate. We attribute this to the fact that LLM only needs to determine the coefficients for different reward APIs, without having to implement detailed functions. On the other hand, Text2Reward yields the highest error rate in code execution. By analyzing the erroneous code, we find that LLM often uses undefined variables or generates code that does not follow the correct template. The execution error rate of CARD is lower than Eureka, demonstrating that our method is more token-efficient than Eureka without redundant parallel sampling. Detailed execution error rate results for LLM-designed reward functions across different algorithms are reported in Table C.10.

C.2. Different LLM APIs (Q6)

To assess the impact of different LLM APIs on CARD, we conduct experiments using `GPT-3.5-turbo-1106` across three tasks: *Door Unlock*, *Drawer Open*, and *Handle Press Side* from Meta-World. The results are compared with those generated using `GPT-4-1106-preview`, as shown in Fig. C.9. The results indicate that: **Obs.❸ `GPT-4` has more powerful capabilities in generating reward function code.** `GPT-4` outperforms `GPT-3.5` on all tasks, with a particularly large improvement in *Door Unlock* and *Drawer Open*.

C.3. Token usage details

We report the detailed token consumption for all algorithms as shown in Table C.11. The results indicate that token usage of the same algorithm across different tasks is generally consistent.

C.4. Failure analysis

In experiments, we find that the performance of the generated reward function may degrade after reward introspection. For example, we present the reward function generated in the iteration 2 and 3 in Tables C.12 and C.13, with the latter performing worse than the former. A closer examination of the code reveals that in the iteration 3, the LLM changes the reward coefficient, leading to a significant performance drop. We believe this is expected, as the LLM improves the reward function through a trial-and-error process. With TPE and dynamic feedback, the LLM can improve the reward function over iterations.

Table C.10
Detailed average execution error rate of generated code among different algorithms.

Method	Door Unlock	Drawer Open	Handle Press side	Handle Press	Sweep Into	Window Open	Average
L2R	0.06 ± 0.24	0.06 ± 0.24	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.02
Text2Reward	0.88 ± 0.33	0.31 ± 0.46	0.44 ± 0.50	0.63 ± 0.48	0.31 ± 0.46	0.31 ± 0.46	0.48
Eureka	0.32 ± 0.23	0.18 ± 0.29	0.23 ± 0.25	0.30 ± 0.30	0.11 ± 0.12	0.29 ± 0.27	0.24
CARD	0.40 ± 0.49	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.07
Method	Lift Cube	Open **CabinetDoor**	Open CabinetDrawer	Pick Cube	Push Chair	Turn Faucet	Average
L2R	0.00 ± 0.00	0.00 ± 0.00	0.31 ± 0.46	0.13 ± 0.33	0.13 ± 0.33	0.06 ± 0.24	0.11
Text2Reward	0.44 ± 0.50	0.25 ± 0.43	0.50 ± 0.50	0.44 ± 0.50	0.63 ± 0.48	0.56 ± 0.50	0.47
Eureka	0.07 ± 0.10	0.21 ± 0.28	0.09 ± 0.16	0.08 ± 0.07	0.05 ± 0.09	0.11 ± 0.15	0.10
CARD	0.25 ± 0.43	0.25 ± 0.43	0.57 ± 0.49	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.18

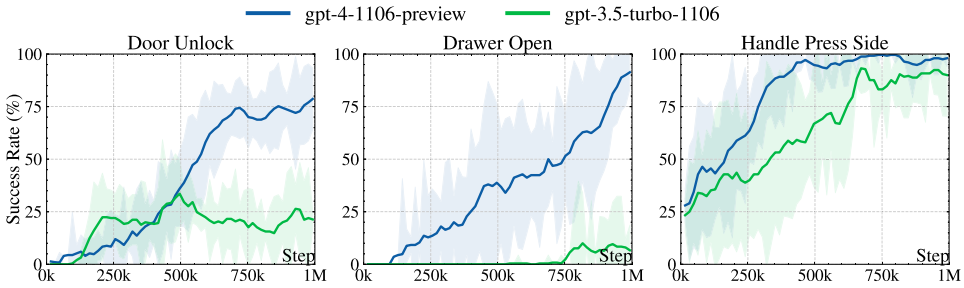


Fig. C.9. Learning curves of CARD on three **Meta-World** tasks using the reward functions generated by different LLM APIs. The solid line represents the mean success rate, and the shaded areas denote the standard deviation, calculated across five random seeds.

Table C.11
Detailed average token usage comparison among different algorithms.

Method	Door Unlock	Drawer Open	Handle Press Side	Handle Press	Sweep Into	Window Open	Average
L2R	2393	2442	2432	2409	2410	2378	2410.7
Text2Reward	1153	1153	1158	1142	1151	1105	1143.7
Eureka	505648	553311	521594	565891	556693	536400	539922.8
CARD	15023	14586	14536	11568	13069	11324	13351.0
Method	Lift Cube	Open CabinetDoor	Open CabinetDrawer	Pick Cube	Push Chair	Turn Faucet	Average
L2R	2300	2348	2278	2206	2163	1969	2210.7
Text2Reward	2321	2344	2328	2263	2277	2294	2304.5
Eureka	689889	1013855	568552	766142	755679	920936	785842.2
CARD	14803	15810	16523	13532	14382	15024	15012.3

C.5. A case where CARD outperforms the oracle

To better understand why CARD performs better than the oracle on certain tasks, we use the Drawer Open task as an example, where CARD outperforms the oracle. We present the oracle reward function and the reward function generated by CARD (in iterations 2) in Tables C.14 and C.15. The comparison shows that CARD introduces more components in its reward function: `action_reg_reward`,`success_reward`, and `time_reward`, which correspond to the penalty for action regularization, the reward for successful task completion, and the penalty for each step, respectively. This more detailed reward design allows CARD to achieve improved performance compared to oracle.

Table C.12
The reward function generated in iteration 2 on ManiSkill2 *Pick Cube* task in Failure Analysis.

```
def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Define the reward components with initial values
    gripper_to_cube_distance_reward = 0.0
    cube_to_goal_distance_reward = 0.0
    action_regularization_reward = 0.0
    grasp_reward = 0.0
    task_completion_reward = 0.0

    # New weights for individual components
    gripper_distance_weight = -1.0  # Reduced the weight to prevent heavy
        initial penalty
    cube_distance_weight = -1.0     # Reduced the weight to prevent heavy
        penalty
    action_reg_weight = -0.001      # Significantly reduced to scale it
        properly
    grasp_weight = 1.0              # Reduced to balance with the task
        completion reward
    completion_weight = 2.0         # New weight for completing the task

    # Calculate the distance between the gripper and cube A
    ee_pose = self.tcp.pose.p
    cube_pose = self.obj.pose.p
    gripper_to_cube_distance = np.linalg.norm(ee_pose - cube_pose)

    # Calculate the distance between cube A and the goal position
    cube_to_goal_distance = np.linalg.norm(cube_pose - self.goal_pos)

    # Regularization of the robot's action
    action_regularization = np.sum(np.square(action))

    # Check if the gripper is grasping the cube
    if self.agent.check_grasp(self.obj):
        # Provide positive reward for successful grasp
        grasp_reward = grasp_weight
        # Calculate the distance reward only if the cube is grasped
        cube_to_goal_distance_reward = cube_distance_weight *
            cube_to_goal_distance
        # Provide additional reward if the cube is at the goal position
        if cube_to_goal_distance < 0.01:  # Threshold for considering the
            task as completed
            task_completion_reward = completion_weight
    else:
        # Calculate the distance reward only if the cube is not grasped
        gripper_to_cube_distance_reward = gripper_distance_weight *
            gripper_to_cube_distance

    # Regularization reward (lower impact than other components)
    action_regularization_reward = action_reg_weight *
        action_regularization

    # Compute the total reward with weights
    total_reward = (gripper_to_cube_distance_reward +
                    cube_to_goal_distance_reward +
                    action_regularization_reward +
                    grasp_reward +
                    task_completion_reward)

    # Create a dictionary for individual reward components
    reward_components = {
        'gripper_to_cube_distance_reward': gripper_to_cube_distance_reward
            ,
        'cube_to_goal_distance_reward': cube_to_goal_distance_reward,
        'action_regularization_reward': action_regularization_reward,
        'grasp_reward': grasp_reward,
        'task_completion_reward': task_completion_reward  # New component
    }

    return total_reward, reward_components
```

Table C.13
The reward function generated in iteration 3 on ManiSkill2 *Pick Cube* task in Failure Analysis.

```
def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Define the reward components with initial values
    gripper_to_cube_distance_reward = 0.0
    cube_to_goal_distance_reward = 0.0
    action_regularization_reward = 0.0
    grasp_reward = 0.0
    completion_reward = 0.0  # Reward for completing the task

    # Weights for individual components
    gripper_distance_weight = -0.5  # Reduced weight to decrease initial
        penalty
    cube_distance_weight = -0.5     # Reduced weight to decrease penalty
    action_reg_weight = -0.0001     # Further reduced to minimize impact
        on initial steps
    grasp_weight = 0.5              # Reduced weight to avoid giving too
        much reward for just grasping
    completion_bonus = 10.0         # Large bonus for completing the task,
         ensuring successful trajectories are highly rewarded

    # Calculate the distance between the gripper and cube A
    ee_pose = self.tcp.pose.p
    cube_pose = self.obj.pose.p
    gripper_to_cube_distance = np.linalg.norm(ee_pose - cube_pose)

    # Calculate the distance between cube A and the goal position
    cube_to_goal_distance = np.linalg.norm(cube_pose - self.goal_pos)

    # Regularization of the robot's action
    action_regularization = np.sum(np.square(action))

    # Check if the gripper is grasping the cube
    if self.agent.check_grasp(self.obj):
        # Provide reward for successfully grasping
        grasp_reward = grasp_weight
        # Calculate the distance reward only if the cube is grasped
        cube_to_goal_distance_reward = cube_distance_weight *
            cube_to_goal_distance
        # Provide a large bonus if the cube is at the goal position
        if cube_to_goal_distance < 0.01:  # Threshold for considering the
            task as completed
            completion_reward = completion_bonus
    else:
        # Calculate the distance reward only if the cube is not grasped
        gripper_to_cube_distance_reward = gripper_distance_weight *
            gripper_to_cube_distance

    # Regularization reward
    action_regularization_reward = action_reg_weight *
        action_regularization

    # Compute the total reward with weights
    total_reward = (gripper_to_cube_distance_reward +
                    cube_to_goal_distance_reward +
                    action_regularization_reward +
                    grasp_reward +
                    completion_reward)

    # Create a dictionary for individual reward components
    reward_components = {
        'gripper_to_cube_distance_reward': gripper_to_cube_distance_reward
            ,
        'cube_to_goal_distance_reward': cube_to_goal_distance_reward,
        'action_regularization_reward': action_regularization_reward,
        'grasp_reward': grasp_reward,
        'completion_reward': completion_reward  # Large bonus for
            completing the task
    }

    return total_reward, reward_components
```

Table C.14
The oracle reward function of Meta-World *Drawer Open* task.

```
def compute_reward(self, action, obs):
    gripper = obs[:3]
    handle = obs[4:7]

    handle_error = np.linalg.norm(handle - self._target_pos)

    reward_for_opening = reward_utils.tolerance(
        handle_error,
        bounds=(0, 0.02),
        margin=self.maxDist,
        sigmoid='long_tail'
    )

    handle_pos_init = self._target_pos + np.array([.0, self.maxDist, .0])
    # Emphasize XY error so that gripper is able to drop down and cage
    # handle without running into it. By doing this, we are assuming
    # that the reward in the Z direction is small enough that the agent
    # will be willing to explore raising a finger above the handle, hook
        it,
    # and drop back down to re-gain Z reward
    scale = np.array([3., 3., 1.])
    gripper_error = (handle - gripper) * scale
    gripper_error_init = (handle_pos_init - self.init_tcp) * scale

    reward_for_caging = reward_utils.tolerance(
        np.linalg.norm(gripper_error),
        bounds=(0, 0.01),
        margin=np.linalg.norm(gripper_error_init),
        sigmoid='long_tail'
    )

    reward = reward_for_caging + reward_for_opening
    reward *= 5.0

    return (
        reward,
        np.linalg.norm(handle - gripper),
        obs[3],
        handle_error,
        reward_for_caging,
        reward_for_opening
    )
```

Table C.15
The reward function generated by CARD in iteration 2 on Meta-World *Drawer Open* task.

```
def compute_dense_reward(self, action: np.ndarray, obs: np.ndarray) ->
    Tuple[float, Dict[str, float]]:
    # Constants for reward shaping
    distance_weight = -2.0  # Emphasize closing the distance to the handle
    goal_state_weight = -2.0  # Increase the weight to emphasize goal
        state achievement
    action_reg_weight = -0.01  # Discourage large actions
    success_bonus = 100.0  # Bonus for successfully opening the drawer
    time_penalty = -0.05  # Reduce penalty to not overshadow the learning
        process

    # Calculate the distance between the end-effector and the drawer
        handle
    distance_to_handle = np.linalg.norm(obs[:3] - obs[4:7])
    distance_reward = distance_weight * distance_to_handle

    # Calculate the difference between the current state of the drawer and
         its goal state
    drawer_state_difference = np.linalg.norm(obs[4:7] - self.env.
        _get_pos_goal())
    goal_state_reward = goal_state_weight * drawer_state_difference

    # Regularization of the robot's action to prevent erratic movements
    action_reg = action_reg_weight * np.sum(np.square(action))

    # Check if the task is successful (drawer is opened)
    # Adjusted the threshold for considering the task successful based on
        empirical evidence
    success_reward = 0.0
    if drawer_state_difference < 0.05:  # Adjust the threshold to a more
        suitable value
        success_reward = success_bonus

    # Time penalty for each step taken
    time_reward = time_penalty * len(obs)

    # Sum up the individual components to get the total reward
    total_reward = distance_reward + goal_state_reward + action_reg +
        success_reward + time_reward

    # Create a dictionary of each individual reward component
    reward_components = {
        'distance_to_handle_reward': distance_reward,
        'goal_state_reward': goal_state_reward,
        'action_reg_reward': action_reg,
        'success_reward': success_reward,
        'time_penalty_reward': time_reward
    }

    return total_reward, reward_components
```

=

Appendix D. License or terms for use

In this section, we provide a summary of the licenses associated with all code and data used in our work.

Benchmarks. Meta-World[2] is under the MIT license. For ManiSkill2,[3] all rigid body environments are licensed under fully permissive licenses (e.g., Apache-2.0). And the soft body environments will follow Warp's license. Currently, they are licensed under the NVIDIA Source Code License for Warp. And the assets are licensed under CC BY-NC 4.0.

Code. We use the open-source implementations of PPO [54] and SAC [55] provided by Raffin et al. [56]. The code[4] is under the MIT license. Our code and data will be publicly available for use under the MIT license.

[2] https://github.com/rlworkgroup/metaworld
[3] https://github.com/haosulab/ManiSkill2
[4] https://github.com/DLR-RM/stable-baselines3

Appendix E. Prompt details

In this section, we provide prompts of CARD, taking Meta-World tasks as an example.

System Prompt and Code Generation Prompt. We refer to the environment abstract description and task description of Meta-World and ManiSkill2 in Xie et al. [26] and make modifications to adapt them to our framework. The system prompt contains the basic instructions for generating a reward function, and Python code that represents the Python class of the environment, the robot, and the rigid object. The system prompt is shown in Table E.16. The task instruction, as shown in Tables B.3 and B.4 is specified by fulfilling `{instruction}`.

Reward Improvement Prompt. To prompt LLM to iteratively refine the reward code, we design the reward improvement prompt, as shown in Table E.17. The reward improvement prompt contains three types of feedback: progress feedback, trajectory feedback, and preference feedback, as described in Section 4.2. If the generated reward code passes TPE, the process feedback and trajectory feedback fulfill `{inference_results}` in the reward improvement prompt, as shown in Table F.30. If the generated code does not pass TPE, the preference feedback will be provided, as shown in Tables F.31 and F.32.

Process feedback includes the average evaluation results of the agent, such as reward, episode length, success rate, and each reward term. Table E.18 shows the process feedback in detail.

Trajectory feedback contains information of the evaluation results after RL training, including step-wise information of the trajectories

Table E.16
System prompt and code generation prompt of Meta-World tasks.

````
You are an expert in robotics, reinforcement learning and code generation.
We are going to use a robot arm to complete given tasks. The action space
    of the robot is a normalized `Box(-1, 1, (4,), float32)`.

Now I want you to help me write a reward function of reinforcement
    learning.
Typically, the reward function of a manipulation task is consisted of
    these following parts (some part is optional, so only include it if
    really necessary):
1. the distance between robot's gripper and our target object
2. difference between current state of object and its goal state
3. regularization of the robot's action
...

class BaseEnv(gym.env):
    self.robot : Robot # the robot in the environment
    self.obj1 : RigidObject # the first object in the environment
    self.obj2 : RigidObject # the second object in the environment, **if
        any**
    self.goal_position : np.ndarray[(3,)] # indicate the 3D position of
        the goal

class Robot:
    self.ee_position : np.ndarray[(3,)] # indicate the 3D position of the
        end-effector
    self.gripper_openness : float # a normalized measurement of how open
        the gripper is, range in [-1, 1]

class RigidObject:
    self.position : np.ndarray[(3,)] # indicate the 3D position of the
        rigid object
    self.quaternion : np.ndarray[(4,)] # indicate the quaternion of the
        rigid object

You are allowed to use any existing python package if applicable. But only
     use these packages when it's really necessary.

I want it to fulfill the following task: {instruction}
1. Please think step by step and tell me what does this task mean;
2. Then write a function that strictly follows the following format, and
    the return value of the function strictly follows the Type Annotation.
`
def compute_dense_reward(self, action: np.ndarray, obs: np.ndarray) ->
    Tuple[float, Dict[str, float]]:
    ...
    return reward, {...}
`
The output of the reward function should consist of two items:
    (1) the total reward,
    (2) a dictionary of each individual reward component.
The code output should be formatted as a python code string: "```python
    ... ```". Just the function body is fine.
3. Do not invent any variable or attribute that is not given.
4. When you writing code, you can also add some comments as your thought.
````

Table E.17
Reward improvement prompt template of Meta-World tasks.

```
I use the reward function you wrote to train the RL agent. The feedback is
     as follows:
{inference_results}
1.Please carefully analyze the above feedback step by step, tell me your
    analysis results.
2.Consider how to enhance the reward function in order to increase the
    sample efficiency of RL and improve the task success rate. Provide a
    new, improved reward function that can better solve the task. Some
    helpful tips for analyzing the feedback:
    (1) If the success rates are always near zero, then you must rewrite
        the entire reward function
    (2) If the values for a certain reward component are near identical
        throughout, then this means RL is not able to optimize this
        component as it is written. You may consider
        (a) Changing its scale or the value of its temperature parameter
        (b) Re-writing the reward component
        (c) Discarding the reward component
    (3) If some reward components' magnitude is significantly larger, then
         you must re-scale its value to a proper range
3. Give the content and reason of the modification in the form of comments
     before the modified position.
4. Do not invent any variable or attribute that is not given. Don't assume
     you can use other information.
```

Table E.18
Process feedback prompt.

```
Evaluation is conducted at specific training intervals. The average
results of the episodes are as follows: When step is {timesteps[i]},
return is {returns[i]}, episode length is {ep_lengths[i]}, success rate is
 {success[i]}, cumulative {sub_reward_name[i]} for the trajectory is {
sub_reward[i]}, ...
```

Table E.19
Process feedback prompt.

```
After training, we evaluated the model and the two trajectories with the
highest and lowest return are as follows:
trajectories with the highest return {highest_return} is: When step is {
steps[i]}, reward is {rewards[i]}, {sub_reward_name[i]} is {sub_reward[i
]}, ... , At step {steps[-1]}, the trajectory ends because {done[i]}.
trajectories with the lowest return {lowest_return} is: When step is {
steps[i]}, reward is {rewards[i]}, {sub_reward_name[i]} is {sub_reward[i
]}, ... , At step {steps[-1]}, the trajectory ends because {done[i]}.
```

with the highest and lowest return, as shown in Table E.19. Since tasks with continuous state spaces may show minimal state changes per step, the sampling interval is treated as a hyperparameter. We find that sampling approximately 5 steps per trajectory provides a good trade-off for constructing Trajectory Feedback. Specifically, we use the formula `log_interval = max_episode_steps // 5`, where `max_episode_steps` is the maximum number of interaction steps for the given task. This ensures that the reward introspection remains efficient without excessive token usage. Additionally, CARD ensures that the first and last steps of each trajectory are recorded, as they provide crucial information about the start and end of the task.

Preference feedback provides the information of sorting the rewards of successful trajectories and failed trajectories, as illustrated in Table E.20.

Table E.20
Process feedback prompt.

```
We collected several trajectories where some agents successfully solved
the task, and others failed. The average reward per step on a successful
trajectory is expected to be higher than that on a failed trajectory.
Using the reward function you provided to calculate the average reward per
 step for each trajectory, We found {success_num} successful trajectories,
 {error_num} of which have a smaller average reward per step than some of
the failed trajectories. This means that the reward function you provided
is {accuracy_rate} accurate in distinguishing successful trajectories.
This indicates a flaw in the design of the reward function. You need to
revise the reward function to ensure that the average reward per step for
most successful trajectories is higher than that of the failed
trajectories.
For example, this is a trajectory where the agent successfully solved the
task, with a return of {success_return}, a length of {success_len}, and an
 average reward per step of {success_reward}: When step is {steps[i]},
reward is {rewards[i]}, {sub_reward_name[i]} is {sub_reward[i]}, ... , At
step {steps[-1]}, the trajectory ends because {done[i]}.
However, the following shows a trajectory where the agent failed to solve
the task, with a return of {failure_return}, a length of {failure_len},
and an average reward per step of {failure_reward}: When step is {steps[i
]}, reward is {rewards[i]}, {sub_reward_name[i]} is {sub_reward[i]}, ... ,
 At step {steps[-1]}, the trajectory ends because {done[i]}.
```

Appendix F. Examples of reward function and reward introspection

F.1. Reward function samples

In this section, to help better understand the introspection capability of CARD and the differences between our method and baselines, we select a challenging task from Meta-World and ManiSkill2 respectively, and provide the reward functions of these methods in the zero-shot setting.

Samples of the reward function designed by CARD (after two rounds of iterations), L2R, Text2Reward and Eureka (after five rounds of iterations) on Meta-World *Door Unlock* task are shown in Tables F.21–F.24, respectively. Samples of the reward function designed by CARD (after two rounds of iterations), L2R, Text2Reward, and Eureka (after five rounds of iterations) on ManiSkill2 *Lift Cube* task are shown in Tables F.25–F.28, respectively.

F.2. Reward introspection results

In order to clearly demonstrate the introspection process of CARD, we select *Lift Cube* task on ManiSkill2 and give the process of 3 rounds of iterations on this task. The iterative process on *Lift Cube* is shown in Tables F.29–F.32. The process demonstrates that CARD iteratively refines the generated reward code based on multiple feedback information.

Table F.21
Sample of the reward function designed by CARD (after two rounds of iterations) on Meta-World *Door Unlock* task.

```
from typing import Tuple, Dict
import numpy as np

def compute_dense_reward(self, action: np.ndarray, obs: np.ndarray) ->
    Tuple[float, Dict[str, float]]:
    # Comment: Reward constants should incentivize successful behavior
        more clearly
    DISTANCE_WEIGHT = -0.1  # Reduced impact to allow other rewards to
        play a role
    ACTION_REGULARIZATION_WEIGHT = -0.01  # Keep this small to avoid
        discouraging exploration
    GRIP_REWARD_BASE = 0.5  # Flat reward for gripping correctly
    GRIP_REWARD_SCALE = 0.5  # Scale for grip reward based on gripper
        openness
    SUCCESS_REWARD = 10.0  # Large reward for successful completion of the
         task

    # Initialize reward and components dictionary
    reward = 0.0
    reward_components = {
        "distance_reward": 0.0,
        "action_regularization": 0.0,
        "grip_reward": 0.0,
        "success_reward": 0.0
    }

    # Calculate distance reward
    distance = np.linalg.norm(obs[:3] - obs[4:7])
    reward_components["distance_reward"] = DISTANCE_WEIGHT * distance

    # Action regularization (reward for using smaller actions)
    reward_components["action_regularization"] =
        ACTION_REGULARIZATION_WEIGHT * np.sum(np.square(action))

    # Grip reward (encouraging the robot to grip when close to the object)
    # Comment: Reward is now given for attempting to grip, scaled by how
        close the gripper is to the correct openness
    is_close = distance < 0.05  # Threshold for "close enough"
    reward_components["grip_reward"] = GRIP_REWARD_BASE if is_close else
        0.0
    reward_components["grip_reward"] += GRIP_REWARD_SCALE * (1.0 - abs(obs
        [3])) if is_close else 0.0

    # Comment: Implement success reward
    # Success reward is given if the task is completed successfully
    # Here we infer success if the object position is close to the goal
        position
    # and the gripper is in a state indicative of "holding" (e.g., not
        fully open or closed)
    goal_reached = np.linalg.norm(obs[4:7] - self.env._get_pos_goal()) <
        0.05
    gripper_holding = 0.2 < obs[3] < 0.8
    if goal_reached and gripper_holding:
        reward_components["success_reward"] = SUCCESS_REWARD

    # Sum up the total reward
    reward = sum(reward_components.values())

    return reward, reward_components
```

Table F.22
Sample of the reward function designed by L2R on Meta-World *Door Unlock* task.

```
import numpy as np
from metaworld.envs import reward_utils
from typing import Tuple

def compute_dense_reward(self, action, obs) -> float:
    def compute_distance_reward(start_pos, end_pos, initial_start_pos,
        initial_end_pos, bounds: Tuple[float, float]) -> float:
        distance_reward = reward_utils.tolerance(
            np.linalg.norm(start_pos - end_pos),
            bounds=bounds,
            margin=np.linalg.norm(initial_start_pos - initial_end_pos),
            sigmoid='long_tail',
        )
        return distance_reward

    import numpy as np

    # Assuming the predefined variables are provided as per the
        environment description
    # self.env.init_tcp, self.env.obj_init_pos, obs[:3],
    # obs[4:7], self.env._get_pos_goal()

    # Calculate the reward for the robotic arm reaching towards the object
    reach_reward = compute_distance_reward(
        start_pos=obs[:3],
        end_pos=obs[4:7],
        initial_start_pos=self.env.init_tcp,
        initial_end_pos=self.env.obj_init_pos,
        bounds=(0, 0.02))

    # Calculate the reward for the object being in place at the goal
        position
    in_place_reward = compute_distance_reward(
        start_pos=obs[4:7],
        end_pos=self.env._get_pos_goal(),
        initial_start_pos=self.env.obj_init_pos,
        initial_end_pos=self.env._get_pos_goal(),
        bounds=(0, 0.02))

    # You can assign different weights to the reach_reward and
        in_place_reward if needed.
    # For simplicity, we will keep both weights as 1 in this example.
    # If you wanted to prioritize one over the other, you could change
        these weights.
    reach_weight = 1.0
    in_place_weight = 1.0

    # Calculate the total reward
    reward = reach_weight * reach_reward + in_place_weight *
        in_place_reward
    return reward
```

Table F.23
Sample of the reward function designed by Text2Reward on Meta-World *Door Unlock* task.

```
import numpy as np
from scipy.spatial.distance import cdist

def compute_dense_reward(self, action, obs) -> float:
    # Define constants for reward tuning
    DISTANCE_WEIGHT = 1.0
    GOAL_REACHED_REWARD = 100.0
    ACTION_PENALTY = 0.1

    # Compute distance between robot's gripper and the lock
    distance = np.linalg.norm(obs[:3] - obs[4:7])

    # Compute difference between current state of object and its goal
        state
    goal_diff = np.linalg.norm(obs[4:7] - self.env._get_pos_goal())

    # Compute action regularization term
    action_penalty = ACTION_PENALTY * np.square(action).sum()

    # Check if the goal has been reached
    goal_reached = cdist(obs[4:7].reshape(1, -1), self.env._get_pos_goal()
        .reshape(1, -1), 'cosine') < 0.01

    # Calculate reward
    reward = - DISTANCE_WEIGHT * distance - goal_diff - action_penalty
    if goal_reached:
        reward += GOAL_REACHED_REWARD

    return reward
```

Table F.24
Sample of the reward function designed by Eureka (after five rounds of iterations) on Meta-World *Door Unlock* task.

```
import numpy as np
from numpy.linalg import norm

def quaternion_to_euler_angle(w, x, y, z):
    ysqr = y * y
    t0 = +2.0 * (w * x + y * z)
    t1 = +1.0 - 2.0 * (x * x + ysqr)
    X = np.arctan2(t0, t1)
    t2 = +2.0 * (w * y - z * x)
    t2 = np.clip(t2, -1.0, 1.0)
    Y = np.arcsin(t2)
    t3 = +2.0 * (w * z + x * y)
    t4 = +1.0 - 2.0 * (ysqr + z * z)
    Z = np.arctan2(t3, t4)
    return X, Y, Z  # Euler angles

def compute_dense_reward(self, action: np.ndarray, obs: np.ndarray) ->
    Tuple[float, Dict[str, float]]:
    # Constants and temperature parameters
    proximity_temp = 0.1
    rotation_temp = 0.1
    proximity_threshold = 0.05
    initial_z_orientation = 0.0  # Assuming we know the initial locked
        orientation of the z-axis

    # Compute distance between end-effector and lock
    distance_to_lock = norm(obs[:3] - obs[4:7])
    # Reward for being close to the lock, normalized to [0, 1]
    proximity_reward = np.exp(-proximity_temp * distance_to_lock) if
        distance_to_lock < proximity_threshold else 0

    # Calculate the current orientation of the lock in Euler angles
    current_w, current_x, current_y, current_z = obs[7:11]
    _, _, current_z_angle = quaternion_to_euler_angle(current_w, current_x
        , current_y, current_z)

    # Calculate the change in the z-axis orientation from the initial
        orientation
    delta_z = np.abs(current_z_angle - initial_z_orientation)
    # Normalize the change to be between 0 and 1, where 1 corresponds to
        the desired rotation
    rotation_normalized = delta_z / np.pi  # Assuming a half-turn is
        sufficient to unlock
    rotation_reward = np.exp(rotation_temp * (1 - rotation_normalized)) if
         delta_z > 0 else 0  # Reward for any change in rotation

    # Combine rewards
    reward = proximity_reward + rotation_reward

    # Reward components breakdown
    reward_components = {
        'proximity_reward': proximity_reward,
        'rotation_reward': rotation_reward
    }

    return reward, reward_components
```

Table F.25
Sample of the reward function designed by CARD (after two rounds of iterations) on ManiSkill2 *Lift Cube* task.

```
def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Decrease the penalty for distance to encourage exploration without
        too much punishment.
    distance_weight = -1.0

    # Significantly increase the reward for lifting to make it much more
        significant.
    # This helps to clearly differentiate between successful and
        unsuccessful attempts.
    lifting_weight = 10.0

    # Slightly decrease the penalty for actions to prevent the agent from
        being too conservative.
    action_weight = -0.05

    # No change to the grasp reward as the agent seems to learn grasping
        well.
    grasp_weight = 0.5

    # Step 1: Calculate the distance between the robot's gripper and cube
        A
    ee_pose = self.tcp.pose
    cube_pose = self.obj.pose
    distance = np.linalg.norm(ee_pose.p - cube_pose.p)
    distance_reward = distance_weight * distance

    # Step 2: Check if the cube is lifted by 0.2 meters
    # The lifting reward is now proportional to the height but has a much
        larger weight.
    lifting_reward = lifting_weight * max(0.0, cube_pose.p[2] - 0.02)

    # Step 3: Apply regularization to the robot's action
    action_penalty = action_weight * np.linalg.norm(action)

    # Step 4: Check if the robot is grasping the cube
    grasp_reward = grasp_weight if self.agent.check_grasp(self.obj) else
        0.0

    # Calculate total reward
    total_reward = distance_reward + lifting_reward + action_penalty +
        grasp_reward

    # Return the total reward and the dictionary of individual reward
        components
    return total_reward, {
        'distance_reward': distance_reward,
        'lifting_reward': lifting_reward,
        'action_penalty': action_penalty,
        'grasp_reward': grasp_reward,
    }
```

Table F.26
Sample of the reward function designed by L2R on ManiSkill2 *Lift Cube* task.

```
import numpy as np

def compute_dense_reward(self, action, info) -> float:
    def compute_reaching_reward(arm_pos, obj_pos, reward_bound: float,
        reward_scaling: float) -> float:
        dist = np.linalg.norm(obj_pos - arm_pos)
        reaching_reward = reward_bound - np.tanh(reward_scaling * dist)
        return reaching_reward

    def compute_grasping_reward(max_angle: int, reward_value: float) ->
        float:
        is_grasped = self.agent.check_grasp(self.obj, max_angle=max_angle)
        grasping_reward = 0.0
        if is_grasped:
            grasping_reward += reward_value
        return grasping_reward

    def compute_lifting_reward(max_angle: int, obj_pos, reward_bound:
        float) -> float:
        is_grasped = self.agent.check_grasp(self.obj, max_angle=max_angle)
        lifting_reward = 0.0
        if is_grasped:
            lifting_reward = obj_pos[2] - self.cube_half_size[2]
            lifting_reward = min(lifting_reward / self.goal_height,
                reward_bound)
        return lifting_reward

    def is_success() -> bool:
        return info["success"]


    import numpy as np

    # Constants for reward calculation
    REACHING_REWARD_BOUND = 1.0
    REACHING_REWARD_SCALING = 5.0
    GRASPING_REWARD_VALUE = 2.0
    LIFTING_REWARD_BOUND = 1.0
    MAX_GRASP_ANGLE = 30
    LIFT_HEIGHT_REWARD_MULTIPLIER = 3.0  # A weight to increase the
        importance of lifting

    # Checking if the task is successfully completed
    if is_success():
        reward = 5
    else:
        # Compute the reaching reward based on the current position of the
            arm and the object
        reaching_reward = compute_reaching_reward(self.tcp.pose.p, self.
            obj.pose.p, REACHING_REWARD_BOUND, REACHING_REWARD_SCALING)

        # Compute the grasping reward if the arm is close enough to grasp
            the object
        grasping_reward = compute_grasping_reward(MAX_GRASP_ANGLE,
            GRASPING_REWARD_VALUE)

        # Compute the lifting reward if the object is grasped and lifted
            to the desired height
        lifting_reward = compute_lifting_reward(MAX_GRASP_ANGLE, self.obj.
            pose.p, LIFTING_REWARD_BOUND)

        # Combine the rewards with respective weights
        reward = reaching_reward + grasping_reward + (lifting_reward *
            LIFT_HEIGHT_REWARD_MULTIPLIER)
    return reward
```

Table F.27
Sample of the reward function designed by Text2Reward on ManiSkill2 *Lift Cube* task.

```
import numpy as np
from scipy.spatial import distance

def compute_dense_reward(self, action) -> float:
    # Define reward weights
    weight_dist = 0.4
    weight_lift = 0.4
    weight_grasp = 0.2

    # Initialize reward
    reward = 0.0

    # Stage 1: Approach the cube
    ee_pos = self.tcp.pose.p
    cube_pos = self.obj.pose.p
    dist_to_cube = np.linalg.norm(ee_pos - cube_pos)
    reward_dist = -weight_dist * dist_to_cube

    # Stage 2: Grasp the cube
    grasp_success = self.agent.check_grasp(self.obj, max_angle=30)
    reward_grasp = weight_grasp * grasp_success

    # Stage 3: Lift the cube
    lift_amount = cube_pos[2] - self.goal_height
    reward_lift = -weight_lift * np.abs(lift_amount)

    # Total reward
    reward = reward_dist + reward_grasp + reward_lift

    # Stage 4: Maintain the cube at the goal height
    if self.obj.pose.p[2] >= self.goal_height:
        reward += 0.1 * (self.obj.pose.p[2] - self.goal_height)

    # Regularize the robot's action
    # We don't want robot to take very big action, so we add a negative
        reward here
    reward -= 0.01 * np.linalg.norm(action)

    return reward
```

Table F.28
Sample of the reward function designed by Eureka (after five rounds of iterations) on ManiSkill2 *Lift Cube* task.

```
import numpy as np
from typing import Tuple

def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Constants and parameters
    distance_reward_temp = 2.0  # Increased temperature for distance
        reward for a smoother penalty
    gripper_reward_scale = 1.0  # Increased scale for gripper reward
    lifting_reward_scale = 2.0  # Increased scale for lifting reward

    # Compute distance between the gripper and cube A
    ee_position = self.tcp.pose.p
    cube_position = self.obj.pose.p
    distance_to_cube = np.linalg.norm(ee_position - cube_position)
    # Adjust the distance reward function to be less steep
    distance_reward = -np.exp(distance_to_cube / distance_reward_temp)

    # Check if the gripper is grasping the cube A and provide reward
    gripper_reward = float(self.agent.check_grasp(self.obj)) *
        gripper_reward_scale

    # Compute lifting reward based on the cube's height above the initial
        position
    lifting_height = np.clip(cube_position[2] - 0.02, 0, np.inf)
    lifting_reward = (lifting_height / self.goal_height) *
        lifting_reward_scale
    # Ensure the lifting reward is within [0, lifting_reward_scale]
    lifting_reward = np.clip(lifting_reward, 0, lifting_reward_scale)

    # Combine the rewards
    total_reward = distance_reward + lifting_reward + gripper_reward

    rewards_dict = {
        "distance_reward": distance_reward,
        "lifting_reward": lifting_reward,
        "gripper_reward": gripper_reward
    }

    return total_reward, rewards_dict
```

Table F.29
The code first generated on *Lift Cube*.

```
def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Initialize reward components
    distance_reward = 0.0
    lifting_reward = 0.0
    action_penalty = 0.0
    grasp_reward = 0.0
    total_reward = 0.0

    # Parameters for reward shaping
    distance_weight = -0.5  # Penalize the distance between gripper and
        cube A
    lifting_weight = 1.0    # Reward for lifting the cube to the desired
        height
    action_weight = -0.1    # Penalize large actions to encourage smooth
        movements
    grasp_weight = 0.5      # Encourage the robot to grasp the cube

    # Step 1: Calculate the distance between the robot's gripper and cube
        A
    ee_pose = self.tcp.pose
    cube_pose = self.obj.pose
    distance = np.linalg.norm(ee_pose.p - cube_pose.p)
    distance_reward = distance_weight * distance

    # Step 2: Check if the cube is lifted by 0.2 meters
    if cube_pose.p[2] >= self.goal_height:
        lifting_reward = lifting_weight

    # Step 3: Apply regularization to the robot's action
    action_penalty = action_weight * np.linalg.norm(action)

    # Step 4: Check if the robot is grasping the cube
    if self.agent.check_grasp(self.obj):
        grasp_reward = grasp_weight

    # Calculate total reward
    total_reward = distance_reward + lifting_reward + action_penalty +
        grasp_reward

    # Return the total reward and the dictionary of individual reward
        components
    return total_reward, {
        'distance_reward': distance_reward,
        'lifting_reward': lifting_reward,
        'action_penalty': action_penalty,
        'grasp_reward': grasp_reward,
    }
```

Table F.30
The code generated after 1 introspection iteration on *Lift Cube*.

```
I use the reward function you wrote to train the RL agent. The feedback is
     as follows:
# Process Feedback:
1.Evaluation is conducted at specific training intervals. The average
    results of the episodes are as follows:
When step is 192000, ......
# Trajectory Feedback
2.After training, we evaluated the model and the two trajectories with the
     highest and lowest return are as follows: ......
......
# LLM Response
Analyzing the feedback:
1. The success rate is consistently 0%, indicating that the RL agent never
     successfully completes the task of lifting cube A by 0.2 meters
    within the given steps.
2. The grasp reward seems to be the only component contributing positively
     to the reward, and the agent appears to have learned to grasp the
    cube reliably.
3. The distance reward is negative in all cases and varies across
    evaluations. It seems that the agent does not consistently minimize
    the distance or that the distance calculation is not reflecting the
    desired behavior.
4. The action penalty seems to be functioning, penalizing larger actions,
    but it might be too small compared to the grasp reward, which could
    lead to the agent preferring to grasp without lifting.
......
def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Parameters for reward shaping
    distance_weight = -2.0  # Increase the magnitude to encourage
        minimizing the distance more effectively
    lifting_weight = 2.0    # Increase the reward for lifting to make it
        more significant than the grasp reward
    action_weight = -0.1    # Keep the penalty for large actions
    grasp_weight = 0.5      # The agent seems to learn grasping well, no
        change needed
    # Step 1: Calculate the distance between the robot's gripper and cube
        A
    ee_pose = self.tcp.pose
    cube_pose = self.obj.pose
    distance = np.linalg.norm(ee_pose.p - cube_pose.p)
    distance_reward = distance_weight * distance
    # Step 2: Check if the cube is lifted by 0.2 meters
    # Modify the lifting reward to be proportional to the height achieved
    lifting_reward = lifting_weight * max(0.0, cube_pose.p[2] - 0.02)
    # Step 3: Apply regularization to the robot's action
    action_penalty = action_weight * np.linalg.norm(action)
    # Step 4: Check if the robot is grasping the cube
    grasp_reward = grasp_weight if self.agent.check_grasp(self.obj) else
        0.0
    # Calculate total reward
    total_reward = distance_reward + lifting_reward + action_penalty +
        grasp_reward
    return total_reward, {
        'distance_reward': distance_reward,
        'lifting_reward': lifting_reward,
        'action_penalty': action_penalty,
        'grasp_reward': grasp_reward,
    }
```

Table F.31
The code generated after 2 introspection iteration on *Lift Cube*.

```
......
# Preference Feedback:
I use the reward function you wrote to train the RL agent. The feedback is
     as follows:
We collected several trajectories where some agents successfully solved
    the task, and others failed.
The average reward per step on a successful trajectory is expected to be
    higher than that on a failed trajectory.
......
# LLM Response
Analyzing the feedback:
1. The successful trajectory has an overall negative return due to the
    accumulation of negative rewards in the early steps, while the failed
    trajectory has a smaller negative return because it consistently
    receives a slightly positive reward throughout. This suggests that the
     negative components of the reward function (i.e., distance_reward and
     action_penalty) might be too punishing, or the positive rewards are
    not significant enough to outweigh the penalties.
2. In the successful trajectory, the lifting_reward is only received at
    the last steps, which is not enough to compensate for the accumulated
    penalties. This means that the reward for the main objective (lifting
    the cube) is not sufficiently high to differentiate successful from
    unsuccessful attempts.
3. The grasp_reward is consistently received in both successful and failed
     trajectories, indicating that the agent has learned to grasp the cube
     but not necessarily to lift it. This could be overshadowing the
    importance of the lifting reward.
Based on these observations, the reward function should be modified to:
- Increase the reward for the main task (lifting the cube).
- Decrease the penalties to a level that encourages exploration without
    too much punishment.
- Ensure that the reward function clearly distinguishes between successful
     and unsuccessful attempts.

def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Decrease the penalty for distance to encourage exploration without
        too much punishment.
    distance_weight = -1.0
    # Significantly increase the reward for lifting to make it much more
        significant.
    # This helps to clearly differentiate between successful and
        unsuccessful attempts.
    lifting_weight = 10.0
    # Slightly decrease the penalty for actions to prevent the agent from
        being too conservative.
    action_weight = -0.05
    # No change to the grasp reward as the agent seems to learn grasping
        well.
    grasp_weight = 0.5
    # Step 1: Calculate the distance between the robot's gripper and cube
        A
    ee_pose = self.tcp.pose
    cube_pose = self.obj.pose
    distance = np.linalg.norm(ee_pose.p - cube_pose.p)
    distance_reward = distance_weight * distance
    # Step 2: Check if the cube is lifted by 0.2 meters
    # The lifting reward is now proportional to the height but has a much
        larger weight.
    lifting_reward = lifting_weight * max(0.0, cube_pose.p[2] - 0.02)
    # Step 3: Apply regularization to the robot's action
    action_penalty = action_weight * np.linalg.norm(action)
    # Step 4: Check if the robot is grasping the cube
    grasp_reward = grasp_weight if self.agent.check_grasp(self.obj) else
        0.0
    # Calculate total reward
    total_reward = distance_reward + lifting_reward + action_penalty +
        grasp_reward
    return total_reward, {
        'distance_reward': distance_reward,
        'lifting_reward': lifting_reward,
        'action_penalty': action_penalty,
        'grasp_reward': grasp_reward,
    }
```

Table F.32
The code generated after 3 introspection iteration on *Lift Cube*.

```
......
# Preference Feedback:
I use the reward function you wrote to train the RL agent. The feedback is
     as follows:
We collected several trajectories where some agents successfully solved
    the task, and others failed.
The average reward per step on a successful trajectory is expected to be
    higher than that on a failed trajectory.
......
# LLM Response
Analyzing the feedback:
1. The successful trajectory has a negative average reward per step
    despite completing the task. This indicates that the penalties (
    distance_reward and action_penalty) are too severe or that the
    lifting_reward is not substantial enough to offset these penalties
    across all steps, especially early in the episode before the cube is
    lifted.
2. The failed trajectory has a smaller negative return because it
    consistently receives a positive grasp_reward that outweighs the
    penalties from distance_reward and action_penalty. This suggests that
    the grasp_reward may be too high relative to the penalties, allowing
    failed trajectories to accumulate higher rewards simply by grasping
    the cube without completing the task.
3. The agent receives a high lifting_reward only at the end of the
    successful trajectory. While it is important for the lifting_reward to
     be significant, it must also encourage incremental progress towards
    lifting the cube, or the agent may not be guided effectively through
    the intermediate steps of the task.

- The grasp_reward should be reduced to prevent it from dominating the
    reward signal.
- The lifting_reward should be fine-tuned to provide incremental rewards
    as the cube is lifted, rather than a large reward only at the goal
    height.
- The penalties should be adjusted to ensure that they encourage the
    correct behaviors without being too harsh.

def compute_dense_reward(self, action: np.ndarray) -> Tuple[float, Dict[
    str, float]]:
    # Reduce the penalty for distance to encourage exploration.
    distance_weight = -0.5  # Reduced severity as the agent was being
        penalized too much for distance.
    # Increase the reward for lifting. Provide incremental reward based on
         the height achieved,
    # not just on reaching the goal height. This will encourage progress
        towards lifting the cube.
    lifting_reward_multiplier = 5.0  # Increased to provide a stronger
        signal for lifting progress.
    # Reduce the penalty for actions to prevent the agent from being too
        conservative.
    action_weight = -0.02  # Reduced severity to encourage more
        exploratory actions.
    # Reduce the grasp reward as it was overshadowing other components.
    grasp_weight = 0.2  # Reduced to balance the overall reward structure.
    # Step 1: Calculate the distance between the robot's gripper and cube
        A
    ee_pose = self.tcp.pose
    cube_pose = self.obj.pose
    distance = np.linalg.norm(ee_pose.p - cube_pose.p)
    distance_reward = distance_weight * distance
    # Step 2: Check the height of the cube and provide incremental lifting
         reward
    lifting_progress = max(0.0, cube_pose.p[2] - 0.02)
    lifting_reward = lifting_reward_multiplier * lifting_progress
    # Step 3: Apply regularization to the robot's action
    action_penalty = action_weight * np.linalg.norm(action)
    # Step 4: Check if the robot is grasping the cube
    grasp_reward = grasp_weight if self.agent.check_grasp(self.obj) else
        0.0
    # Calculate total reward
    total_reward = distance_reward + lifting_reward + action_penalty +
        grasp_reward
    # Return the total reward and the dictionary of individual reward
        components
    return total_reward, {
        'distance_reward': distance_reward,
        'lifting_reward': lifting_reward,
        'action_penalty': action_penalty,
        'grasp_reward': grasp_reward,
    }
```

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, Nature 518(7540) (2015) 529–533.

[2] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, Nature 529(7587) (2016) 484–489.

[3] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, et al., Dota 2 with large scale deep reinforcement learning, (2019). arXiv preprint arXiv:1912.06680

[4] R.S. Sutton, Reinforcement learning: an introduction, MIT press Cambridge 1 (2018).

[5] A.Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: Theory and application to reward shaping, in: ICML, 99, Citeseer, 1999, pp. 278–287.

[6] E. Wiewiora, Potential-based shaping and q-value initialization are equivalent, J. Artif. Intell. Res. 19 (2003) 205–208.

[7] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, S. Levine, Meta-World: a benchmark and evaluation for multi-task and meta reinforcement learning, in: Conference on Robot Learning (CoRL), 100, PMLR, 2020, pp. 1094–1100.

[8] O.M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al., Learning dexterous in-hand manipulation, Int. J. Robot. Res. 39(1) (2020) 3–20.

[9] W.B. Knox, A. Allievi, H. Banzhaf, F. Schmitt, P. Stone, Reward (mis) design for autonomous driving, Artif. Intell. 316 (2023) 103829.

[10] A.D. Laud, Theory and Application of Reward Shaping in Reinforcement Learning, University of Illinois at Urbana-Champaign, 2004.

[11] A. Gupta, A. Pacchiano, Y. Zhai, S. Kakade, S. Levine, Unpacking reward shaping: understanding the benefits of reward engineering on sample complexity, in: Advances in Neural Information Processing Systems (NeurIPS), 35, 2022, pp. 15281–15295.

[12] Y. Shi, H. Dong, C.R. He, Y. Chen, Z. Song, Mixed vehicle platoon forming: a multi-agent reinforcement learning approach, IEEE Internet Things J. 12, (2025) 16886-16898.

[13] H. Ding, W. Zhuang, H. Dong, G. Yin, S. Liu, S. Bai, Eco-driving strategy design of connected vehicle among multiple signalized intersections using constraint-enforced reinforcement learning, IEEE Trans. Transp. Electrif. 11, (2024) 732-743.

[14] S. Booth, W.B. Knox, J. Shah, S. Niekum, P. Stone, A. Allievi, The perils of trial-and-error reward design: misdesign through overfitting and invalid task specifications, in: Proceedings of the AAAI Conference on Artificial Intelligence, 37, 2023, pp. 5920–5929.

[15] D. Hadfield-Menell, S. Milli, P. Abbeel, S.J. Russell, A. Dragan, Inverse reward design, Adv. Neural Inform. Process. Syst. 30 (2017).

[16] B.D. Ziebart, A.L. Maas, J.A. Bagnell, A.K. Dey, et al., Maximum entropy inverse reinforcement learning, in: AAAI, 8, Chicago, IL, USA, 2008, pp. 1433–1438.

[17] M. Wulfmeier, P. Ondruska, I. Posner, Maximum entropy deep inverse reinforcement learning, (2015). arXiv preprint arXiv:1507.04888

[18] C. Finn, S. Levine, P. Abbeel, Guided cost learning: deep inverse optimal control via policy optimization, in: International Conference on Machine Learning, 48, PMLR, 2016, pp. 49–58.

[19] J. Ho, S. Ermon, Generative adversarial imitation learning, in: D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, R. Garnett (Eds.), Advances in Neural Information Processing Systems, 29, Curran Associates, Inc., 2016. https://proceedings.neurips.cc/paper_files/paper/2016/file/cc7e2b878868cbae992d1fb743995d8f-Paper.pdf.

[20] J. Fu, K. Luo, S. Levine, Learning robust rewards with adverserial inverse reinforcement learning, in: International Conference on Learning Representations (ICLR), 2018. https://openreview.net/forum?id=rkHywl-A-.

[21] P.F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, D. Amodei, Deep reinforcement learning from human preferences, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems, 30, Curran Associates, Inc., 2017. https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf.

[22] B. Ibarz, J. Leike, T. Pohlen, G. Irving, S. Legg, D. Amodei, Reward learning from human preferences and demonstrations in atari, Adv. Neural Inform. Process. Syst. 31 (2018).

[23] K. Lee, L.M. Smith, P. Abbeel, PEBBLE: feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training, in: International Conference on Machine Learning (ICML), 139 of *Proceedings of Machine Learning Research*, PMLR, 2021, pp. 6152–6163.

[24] R. Liu, F. Bai, Y. Du, Y. Yang, Meta-Reward-Net: implicitly differentiable reward learning for preference-based reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), 35, 2022, pp. 22270–22284.

[25] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.H. Lee, M.G. Arenas, H.T.L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, F. Xia, Language to rewards for robotic skill synthesis, in: Conference on Robot Learning (CoRL) of Proceedings of Machine Learning Research, 229, PMLR, 2023, pp. 374–404.

[26] T. Xie, S. Zhao, C.H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, T. Yu, Text2reward: reward shaping with language models for reinforcement learning, in: The Twelfth International Conference on Learning Representations, 2024. https://openreview.net/forum?id=tUM39YTRxH.

[27] Y.J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, A. Anandkumar, Eureka: human-level reward design via coding large language models, in: The Twelfth International Conference on Learning Representations, 2024. https://openreview.net/forum?id=IEduRUO55F.

[28] H. Li, X. Yang, Z. Wang, X. Zhu, J. Zhou, Y. Qiao, X. Wang, H. Li, L. Lu, J. Dai, Auto mc-reward: automated dense reward design with large language models for minecraft, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 16426–16435.

[29] S. Singh, R.L. Lewis, A.G. Barto, Where do rewards come from, in: Proceedings of the Annual Conference of the Cognitive Science Society, Cognitive Science Society, 2009, pp. 2601–2606.

[30] Y. Luo, z. jiang, S. Cohen, E. Grefenstette, M.P. Deisenroth, Optimal transport for offline imitation learning, in: The Eleventh International Conference on Learning Representations, 2023. https://openreview.net/forum?id=MhuFzFsrfvH.

[31] J. Lyu, X. Ma, L. Wan, R. Liu, X. Li, Z. Lu, SEABO: a simple search-based method for offline imitation learning, in: The Twelfth International Conference on Learning Representations, 2024. https://openreview.net/forum?id=MNyOI3C7YB.

[32] R. Liu, Y. Du, F. Bai, J. Lyu, X. Li, PEARL: zero-shot cross-task preference alignment and robust reward learning for robotic manipulation, in: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, F. Berkenkamp (Eds.), Proceedings of the 41st International Conference on Machine Learning, of Proceedings of Machine Learning Research, 235, PMLR, 2024, pp. 30946–30964. https://proceedings.mlr.press/v235/liu24o.html.

[33] Y.J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, A. Zhang, VIP: towards universal visual reward and representation via value-implicit pre-training, in: The Eleventh International Conference on Learning Representations, 2023. https://openreview.net/forum?id=YJ7o2wetJ2.

[34] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D.A. Huang, Y. Zhu, A. Anandkumar, Minedojo: building open-ended embodied agents with internet-scale knowledge, Adv. Neural Inform. Process. Syst. 35 (2022) 18343–18362.

[35] Y.J. Ma, V. Kumar, A. Zhang, O. Bastani, D. Jayaraman, Liv: language-image representations and rewards for robotic control, in: International Conference on Machine Learning, 202, PMLR, 2023, pp. 23301–23320.

[36] Y. Du, K. Konyushkova, M. Denil, A. Raju, J. Landon, F. Hill, N. de Freitas, S. Cabi, et al., Vision-language models as success detectors, in: S. Chandar, R. Pascanu, H. Sedghi, D. Precup (Eds.), Proceedings of The 2nd Conference on Lifelong Learning Agents, of Proceedings of Machine Learning Research, 232, PMLR, 2023, pp. 120–136. https://proceedings.mlr.press/v232/du23b.html.

[37] S. Karamcheti, S. Nair, A.S. Chen, T. Kollar, C. Finn, D. Sadigh, P. Liang, Language-driven representation learning for robotics, (2023). arXiv preprint arXiv:2302.12766

[38] Y. Du, O. Watkins, Z. Wang, C. Colas, T. Darrell, P. Abbeel, A. Gupta, J. Andreas, Guiding pretraining in reinforcement learning with large language models, in: International Conference on Machine Learning (ICML), of Proceedings of Machine Learning Research, 202, PMLR, 2023, pp. 8657–8677.

[39] M. Kwon, S.M. Xie, K. Bullard, D. Sadigh, Reward design with language models, in: The Eleventh International Conference on Learning Representations, 2023. https://openreview.net/forum?id=10uNUgI5Kl.

[40] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, A. Zeng, Code as policies: language model programs for embodied control, in: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2023, pp. 9493–9500.

[41] S. Huang, Z. Jiang, H. Dong, Y. Qiao, P. Gao, H. Li, Instruct2act: mapping multi-modality instructions to robotic actions with large language model, (2023). arXiv preprint arXiv:2305.11176

[42] H. Wang, G. Gonzalez-Pumariega, Y. Sharma, S. Choudhury, Demo2code: from summarizing demonstrations to synthesizing code via extended chain-of-thought, in: Proceedings of the 37th International Conference on Neural Information Processing Systems, 36, 2023, pp. 14848–14956.

[43] K. Lin, C. Agia, T. Migimatsu, M. Pavone, J. Bohg, Text2motion: from natural language instructions to feasible plans, Autonom. Robots 47(8) (2023) 1345–1365.

[44] Y. Ding, X. Zhang, C. Paxton, S. Zhang, Task and motion planning with large language models for object rearrangement, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2023, pp. 2086–2092.

[45] Y. Xie, C. Yu, T. Zhu, J. Bai, Z. Gong, H. Soh, Translating natural language to planning goals with large-language models, (2023). arXiv preprint arXiv:2302.05128

[46] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, P. Stone, Llm+ p: empowering large language models with optimal planning proficiency, (2023). 2304.11477

[47] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, A. Garg, Progprompt: generating situated robot task plans using large language models, in: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2023, pp. 11523–11530.

[48] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, Voyager: an open-ended embodied agent with large language models, Trans. Mach. Learn. Res. (2024). https://openreview.net/forum?id=ehfRiF0R3a.

[49] T. Silver, S. Dan, K. Srinivas, J.B. Tenenbaum, L. Kaelbling, M. Katz, Generalized planning in pddl domains with pretrained large language models, Proc. AAAI Conf. Artif. Intell. 38(18) (2024) 20256–20264.

[50] H. Le, Y. Wang, A.D. Gotmare, S. Savarese, S.C.H. Hoi, Coderl: mastering code generation through pretrained models and deep reinforcement learning, Adv. Neural Inform. Process. Syst. 35 (2022) 21314–21328.

[51] T.X. Olausson, J.P. Inala, C. Wang, J. Gao, A. Solar-Lezama, Demystifying gpt self-repair for code generation, (2023). arXiv preprint arXiv:2306.09896

[52] J. Gu, F. Xiang, X. Li, Z. Ling, X. Liu, T. Mu, Y. Tang, S. Tao, X. Wei, Y. Yao, X. Yuan, P. Xie, Z. Huang, R. Chen, H. Su, Maniskill2: a unified benchmark for generalizable manipulation skills, in: The Eleventh International Conference on Learning Representations, 2023. https://openreview.net/forum?id=b_CQDy9vrD1.
[53] OpenAI, GPT-4, 2023. https://openai.com/research/gpt-4.
[54] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, (2017). arXiv preprint arXiv:1707.06347
[55] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International Conference on Machine Learning, 80, PMLR, 2018, pp. 1861–1870.
[56] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-baselines3: reliable reinforcement learning implementations, J. Mach. Learn. Res. 22(268) (2021) 1–8.